

Project Report

AttackDNA

Organization-Specific Behavioral Detection Engine

Name: Amber Waseem

Reference Code: ARZ-2026-UDEB

Prepared by: Amber

Submitted to: Wardet Wahaj

GitHub Repository:

<https://github.com/Amberr8/attackdna>

Part I: Previous Project on CICIDS2017

1. Introduction

1.1 Problem Statement

Traditional intrusion detection systems rely on generic, static rules, for example Sigma, Snort, or YARA signatures, that apply the same definition of suspicious activity to every host on a network. This causes two well known problems in real Security Operations Centers. First, high false positive rates: a rule tuned for a quiet workstation will constantly misfire on a busy file server, and vice versa. Second, blind spots to novel attacks: signature based tools can only detect what they already have a rule for.

AttackDNA addresses this by learning a behavioral baseline per entity rather than applying one global rule set, so that normal is defined relative to each host's own historical behavior.

1.2 Objectives

- Build a working, genuinely trained machine learning pipeline for anomaly detection, not an API wrapper.
- Detect attacks without ever training on attack labeled data, using a fully unsupervised approach.
- Produce human readable explanations for every flagged anomaly rather than a binary alert.
- Map detected anomalies to standardized MITRE ATT&CK techniques.
- Present results through an interactive analyst facing dashboard.
- Operate entirely on CPU hardware with free, open source tools and no dependency on always on live infrastructure.

2. Dataset

2.1 CICIDS2017 Overview

CICIDS2017, published by the Canadian Institute for Cybersecurity at the University of New Brunswick, is a labeled network traffic dataset captured over five days on a real network testbed. Traffic was processed by CICFlowMeter into flow records, where each row summarizes one network conversation using approximately 79 numeric statistics such as duration, packet counts, byte rates, inter arrival timing, and TCP flag counts, together with a ground truth Label column.

Two versions of this dataset are published. The version initially downloaded for this project, named MachineLearningCSV, excludes IP address columns, which made per entity modeling impossible. This was identified early and corrected by switching to the GeneratedLabelledFlows version, which retains Flow ID, Source IP, Source Port, Destination IP, Destination Port, Protocol, and Timestamp alongside the same flow statistics, for 85 columns in total.

2.2 Days Used

Day / File	Role	Content
Monday-WorkingHours	Baseline, training	100% benign traffic
Tuesday-WorkingHours	Baseline, BENIGN rows only	Brute force present, filtered to BENIGN
Wednesday-WorkingHours	Held out test	DoS Hulk, GoldenEye, Slowloris, Slowhttptest, Heartbleed
Thursday-Morning-WebAttacks	Baseline, BENIGN rows only	Web attacks present, filtered to BENIGN
Thursday-Afternoon-Infiltration	Baseline, BENIGN rows only	Infiltration present, filtered to BENIGN

Day / File	Role	Content
Friday-Morning	Held out test	Botnet traffic
Friday-Afternoon-PortScan	Held out test	Port scanning
Friday-Afternoon-DDoS	Held out test	DDoS traffic

The final training baseline combines Monday, which is fully benign, with the BENIGN labeled rows from Tuesday and both Thursday files, totaling 2,106,675 benign training rows across 15 internal entities. No attack labeled row, and no row from Wednesday or Friday, was included in training, which avoids data leakage between training and evaluation.

2.3 Data Quality Issues Identified and Fixed

- Column headers contain leading whitespace, for example a column literally named ' Flow Duration'. Fixed by stripping all column names on load.
- Flow Bytes per second and Flow Packets per second become infinite when Flow Duration is zero. Fixed by converting infinite values to missing values and dropping the affected rows, a small fraction of the data.
- Some Label values contain a non UTF-8 byte, hex 0x96, a corrupted en dash character. Fixed by normalizing the Label string after load.
- Thursday-Morning-WebAttacks fails to load under default UTF-8 decoding because of this byte. Fixed by reading the file with latin1 encoding.
- Exact duplicate rows exist in the raw files. Removed using pandas drop_duplicates.

3. System Design

3.1 Defining an Entity

A core requirement of AttackDNA is per entity behavioral modeling, but CICIDS2017 does not label which IP address in a flow is the internal host being monitored. CICFlowMeter assigns the Source and Destination columns based on which side sent the first packet of the flow, not based on internal versus external network membership. As a result, a single internal host's traffic is split across both the Source IP and Destination IP columns.

Check	Count	Percent
Flows where Source IP is internal (192.168.10.x)	161,018	80.5%
Flows where Destination IP is internal	89,380	44.7%
Flows where both sides are internal	50,522	25.3%
Flows where neither side is internal (edge case)	124	0.06%

Design decision: an entity column was derived as whichever IP address in the flow belongs to the internal subnet 192.168.10.0/24, regardless of which raw column it appears in. When both sides of a flow are internal, the flow is counted once for each participating entity. Flows where neither side is internal, 0.06 percent of the data, are dropped.

3.2 Pipeline Overview

Core pipeline: data_prep.py, feature_selection.py, train.py, detect.py, evaluate.py, explain.py, mitre_map.py, dashboard/app.py.

Extended pipeline, added for supervisor requested improvements: time_window_features.py, diagnose_entity.py, scan_detector.py, combine_detectors.py, diagnose_fpr.py, train_lof.py, detect_lof.py, seed_stability.py.

4. Core Pipeline Implementation (CICIDS2017 version)

The original, CICIDS2017 specific source code for data_prep.py, feature_selection.py, train.py, detect.py, explain.py, and mitre_map.py is superseded by the GeNIS adapted versions documented in full in Part II, and is not duplicated here. Part II documents every script's current implementation in full, including exactly what changed relative to this original version and why. The CICIDS2017 results below remain the reference point this project is validated against.

5. Initial Results, Before Supervisor Review

The first full pipeline run trained 15 per entity models on Monday's benign traffic only, then evaluated against Wednesday's DoS traffic.

True Label	Flagged / Total	Detection Rate
BENIGN	36,111 / 648,986	5.6% (false positive rate)
DoS Hulk	153,905 / 230,123	66.9%
DoS GoldenEye	6,779 / 10,293	65.9%
DoS slowloris	3,012 / 5,796	52.0%
DoS Slowhttptest	4,005 / 5,499	72.8%
Heartbleed	11 / 11	100.0%

This followed a threshold calibration fix: the default contamination='auto' setting flagged roughly 10 percent of traffic regardless of how clean it was, producing an inflated 8.8 percent false positive rate. Calibrating a custom per entity threshold targeting a 5 percent false positive rate on held out validation data reduced this to 5.6 percent.

Entity 192.168.10.50, the CICIDS2017 web server and primary DoS target, showed an elevated false positive rate of 18.5 percent compared to the network average of 5.6 percent, even after threshold calibration. This entity had the second smallest Monday baseline, 11,307 flows, and was the sole target of sustained DoS attacks throughout Wednesday.

6. Limitations Identified and Addressed

#	Limitation	Status
1	Limited baseline data, one day only	Addressed, expanded to four days, mixed result
2	Static baseline, never updates	Addressed by design, retraining process defined
3	Same threshold for every host	Addressed, calibrated per entity
4	Only one ML algorithm tested	Addressed, Section 7.2
5	No feature selection	Addressed, 79 to 57 features
6	Weak MITRE mapping logic	Addressed, evidence based scoring across all features
7	Explanations are statistical only	Partially addressed, model native explanations remain future work
8	No real time detection	Documented as future work
9	Tested on only one day, one attack type	Addressed, expanded to two days, five attack types
10	Risk score used only one factor	Addressed, weighted four factor score

#	Limitation	Status
11	No multi flow pattern detection	Proven limitation, directly addressed in Section 7.1
12	No formal accuracy metrics	Addressed, precision, recall, F1 added

The combined baseline, expanded to four days, grew to 2,106,675 rows across all 15 entities. The result was mixed: network wide false positive rate barely changed, from 5.6 to 6.0 percent, and entity .50's false positive rate actually rose slightly, from 18.5 to 21.8 percent, genuine evidence that more historical data alone does not close the gap to an unseen test day.

7. Supervisor-Requested Improvements

After reviewing the initial results, the supervisor requested four items in priority order: add time window features to close the Bot and PortScan detection gap, benchmark Local Outlier Factor for comparison, validate results across multiple data splits, and diagnose the root cause of the baseline false positive rate.

7.1 Time Window Features

Prior evaluation showed 0.2 percent recall on PortScan and 2.2 percent recall on Bot traffic, versus 63 to 84 percent recall on DoS and DDoS. Two engineered features were added: `connections_per_minute` and `distinct_ports_per_minute`, computed per entity in fixed 60 second buckets from the real Timestamp column.

Retraining Isolation Forest directly on these new features produced no meaningful change, PortScan recall stayed at 0.2 percent. The root cause is mechanistic: Isolation Forest isolates scattered outliers quickly, but PortScan flows form a large, dense, self similar cluster, which requires many additional splits to separate internally, producing a low anomaly score despite being obviously abnormal from a security standpoint. This is a documented structural weakness of Isolation Forest, not a tuning failure.

A dedicated rule based scan detector was built instead, flagging a flow when either time window feature exceeds the entity's benign mean plus five standard deviations. Standalone, this achieved 99.1 percent PortScan recall with a 0.7 percent false positive rate. Combined with Isolation Forest, DoS recall improved from 68.3 to 98.1 percent, F1 0.743 to 0.913; DDoS from 63.6 to 99.3 percent, F1 0.722 to 0.872; PortScan from 0.2 to 99.2 percent, F1 0.003 to 0.962. Bot recall remained unchanged at 2.2 percent, since beaconing traffic does not produce a volume burst by design.

7.2 Local Outlier Factor Comparison

LOF was tested with `novelty=True`, the same per entity structure, and the same 5 percent target false positive rate calibration as Isolation Forest, with training capped at 20,000 rows per entity. LOF outperformed Isolation Forest on every attack type tested: DoS F1 0.828 versus 0.743, DDoS 0.855 versus 0.722, PortScan 0.957 versus 0.003 standalone, without needing the manually engineered scan detector. Bot detection remained near zero across all approaches tested.

7.3 Split and Seed Stability Testing

Training and evaluation were repeated with five random seeds. F1 standard deviation was under 1 percent relative variation for DoS and DDoS, confirming the results are stable rather than a fluke of one particular data split.

7.4 Root-Cause Diagnosis of the False Positive Rate

The 5.6 to 6 percent network false positive rate was traced to two entities: 192.168.10.50 at 22.4 percent, attributed to attack adjacent distribution drift on the day it was directly targeted, and 192.168.10.25 at 12.8

percent, attributed to genuinely reduced activity relative to its training baseline. The remaining 13 entities performed close to their 5 percent calibration target.

8. Dashboard

An interactive Streamlit dashboard presents entity level risk scores, MITRE technique distribution, and a filterable alert table. The dashboard has since been rebuilt for the GeNIS deployment; see Part II, Section 22. Screenshots are inserted by the author separately and are not reproduced in this text.

9. Discussion

Two nuances were surfaced during this project. First, per entity relative risk scaling incorrectly gave a zero attack entity Critical alerts, while network wide global scaling correctly fixed this but let a single noisy high volume entity dominate the top of the ranking; the final weighted risk score partially mitigates this by including entity criticality as a separate term. Second, and more broadly useful, the time window features contained an extremely strong statistical signal, roughly 90 standard deviations of separation during the PortScan attack, and yet Isolation Forest still failed to use it, because the attack traffic formed a dense, self similar cluster rather than scattered outliers. Feature engineering and algorithm choice are not independent decisions, and this exact lesson reappears in Part II.

10. Original Conclusion

AttackDNA demonstrates a complete, working, per entity behavioral anomaly detection pipeline on CICIDS2017: data cleaning, per entity model training, calibrated detection, statistical explanation generation, rule based MITRE ATT&CK mapping, and an interactive dashboard. Following supervisor review, the system was extended with time window burst features and a dedicated scan detector, benchmarked against Local Outlier Factor, validated for stability across five random seeds, and had its baseline false positive rate traced to two specific, individually explainable causes.

This CICIDS2017 baseline is retained unchanged as the reference point for Part II, where the same methodology and, largely, the same source code are adapted and re-evaluated against a newer dataset, GeNIS, to test whether the approach generalizes beyond 2017 era traffic.

Part II: Generalization to a Newer Dataset (GeNIS)

11. Motivation and Dataset Selection

11.1 Why Validate Beyond CICIDS2017

CICIDS2017 is now several years old, and the supervisor requested validation of the AttackDNA methodology against newer network traffic, to test whether a per entity, unsupervised approach generalizes beyond the conditions of a single 2017 capture, rather than replacing the original work. The CICIDS2017 results in Part I are kept as the development baseline. This section documents the same methodology applied to GeNIS, a 2025 dataset, and reports the results honestly, including where they are weaker than the original.

11.2 Requirements a Newer Dataset Had to Meet

AttackDNA's entire design depends on per host behavioral modeling, which requires more than a generic labeled flow CSV. A candidate dataset needed source and destination host identifiers to build the entity column, a way to order traffic in time for burst detection, a destination port field, and a clear benign versus attack label with enough benign traffic per host to calibrate a baseline.

11.3 Datasets Considered and Rejected

- CSE-CIC-IDS2018: the closest structural successor to CICIDS2017, but only one year newer, and several publicly mirrored 'cleaned' versions of it, including common Kaggle uploads, remove the Source IP and Destination IP columns entirely as part of a standard preprocessing convention aimed at generic classifiers.
- CICIoT2023: focused on IoT traffic and genuinely newer, but its public CSVs are aggregated over fixed packet windows with no host identifier columns at all. Recovering them would require re-running a flow exporter against the raw packet captures, a separate project on its own.
- NF-CSE-CIC-IDS2018-v2: a NetFlow re-export of 2018 traffic that does retain the standard five tuple, source IP, destination IP, source port, destination port, protocol, and was kept as a fallback candidate, but is still only one year newer than the original dataset.
- BigFlow-NIDS (2025): large and recent, but built by merging four separate testbeds into one file, which conflicts with AttackDNA's assumption of a single coherent internal network, and its column schema after merging was not confirmed to retain host identifiers.
- Generic Kaggle uploads of any of the above: most 'ML ready' Kaggle versions strip IP addresses specifically because they are built for training generic classifiers rather than per host analysis, which is the same problem that affected the original CICIDS2017 MachineLearningCSV download described in Part I, Section 2.1.

11.4 GeNIS: The Dataset Selected

GeNIS is a 2025 network intrusion dataset built on the Airbus CyberRange platform, exported using HERA, a flow exporter built specifically to address known limitations in CICFlowMeter. Investigation of the actual delivered CSV confirmed it retains anonymized integer host identifiers for both sides of every flow, together with source and destination port fields and multiple label granularities. This made it the only newer candidate confirmed, by direct inspection of the data rather than by documentation alone, to support per entity modeling.

12. Dataset Comparison: CICIDS2017 versus GeNIS

Property	CICIDS2017	GeNIS
Capture year	2017	2025
Flow exporter	CICFlowMeter	HERA (Argus based)

Property	CICIDS2017	GeNIS
Raw feature count	79 flow statistics	87 columns, several already one hot encoded
Host identifier fields	Source IP / Destination IP, dotted decimal	Ssaddr / Sdaddr, anonymized integer host IDs
Internal versus external hosts	Real internal subnet, 192.168.10.0/24, versus external traffic	Single closed testbed, no external subnet to filter on
Timestamp	Real wall clock Timestamp column	No real timestamp; only Offset, an Argus record position value
Label structure	Single Label column, BENIGN or named attack	BinaryLabel, CategoryLabel, SubCategoryLabel
Benign baseline structure	Entire clean days with zero attack traffic	Random train/test split; a benign row's neighbors can include attack traffic
Traffic composition	Mostly benign, five distinct attack days	Attack heavy, roughly 93% non benign in the training split
Entity count	15 internal hosts	116 unique host IDs, of which 80 have sufficient benign rows to train on
Total rows (post entity expansion)	Millions across five days	589,688 training rows, 147,424 test rows

Every row marked as a difference above required a corresponding change to the pipeline, documented section by section below.

13. Preprocessing Pipeline Changes

13.1 A Configuration Layer Instead of Hard-Coded Values

Rather than forking the codebase per dataset, a single `config.py` module was introduced. Every script now imports a CFG dictionary holding the column names, the benign label value, the internal subnet prefix where one exists, and dataset specific settings such as the time bucketing strategy. This keeps CICIDS2017 support working unchanged while adding GeNIS support, and is the single place a third dataset would be added in future.

config.py: dataset-specific settings for both GeNIS

```

"""
Central configuration for AttackDNA.

Change DATASET below to switch which dataset-specific settings the rest
of the pipeline uses. Every script that needs a column name, the benign
label value, or the internal-host rule should import CFG from here
instead of hard-coding a value, so the same code works on either dataset.
"""

DATASET = "genis" # "genis" or "cicids2017"

DATASET_CONFIGS = {
    "cicids2017": {
        "src_ip_col": "Source IP",
        "dst_ip_col": "Destination IP",
        "label_col": "Label",
        "benign_value": "BENIGN",
        "timestamp_col": "Timestamp",
        "time_col": None,
        "offset_bucket_size": None,
        "dport_col": "Destination Port",
    }
}

```



```

# CICIDS2017 has real internal (192.168.10.x) vs external hosts.
"internal_prefix": "192.168.10.",
"default_target_entity": "192.168.10.50",
},
"genis": {
    "src_ip_col": "Ssaddr",
    "dst_ip_col": "Sdaddr",
    "label_col": "CategoryLabel",
    "benign_value": "benign",
    # GeNIS has no real wall-clock timestamp column. "Offset" is an
    # Argus record-position value (confirmed empirically: multiple
    # flows lasting 28-40s share the identical Offset value, which
    # is impossible if Offset were elapsed seconds). It's only a
    # rough capture-order proxy, not real time - so time_window_features.py
    # buckets by Offset ranges instead of by real seconds. This is a
    # documented substitution, not literal per-minute bucketing.
    "timestamp_col": None,
    "time_col": "Offset",
    # Chosen empirically: bucket_size=10000 gives clear separation
    # between benign (median 1 distinct port/bucket) and recon/scan
    # traffic (median 21 distinct ports/bucket) on the GeNIS test set.
    "offset_bucket_size": 10000,
    "dport_col": "Dport",
    # GeNIS is a closed testbed; every host ID is "internal", there
    # is no subnet to filter external traffic out with.
    "internal_prefix": None,
    # Entity 1: highest-volume entity in the test set (35,250 rows),
    # heavily attacked (32,076 attack rows) but still has a trained
    # model (some benign baseline) - the closest GeNIS analogue to
    # CICIDS2017's 192.168.10.50 web server case study.
    "default_target_entity": 1,
},
}

```

```
CFG = DATASET_CONFIGS[DATASET]
```

13.2 Entity Definition: No Internal Subnet to Filter On

CICIDS2017's `extract_entity` function filters to the 192.168.10.0/24 subnet before assigning the entity column. GeNIS has no such subnet; Ssaddr and Sdaddr are anonymized integer identifiers inside a single closed testbed, and every host in the capture is effectively internal. The design decision was to treat every unique host ID as its own entity, built from both the source and destination columns of every flow, exactly mirroring the 'count it for both sides' logic used for CICIDS2017's genuinely internal hosts, but without a subnet based filter step. Investigation of the actual data confirmed 85 unique Ssaddr values and 76 unique Sdaddr values, overlapping in 45 of them, for 116 unique entities overall.

data_prep.py: entity extraction now dataset aware, plus a boolean-to-int cast step GeNIS specifically requires

```

import pandas as pd
import numpy as np
import sys

from config import CFG, DATASET

def load_raw_csv(path):
    """Load the CSV and strip whitespace from column names.

    CICIDS2017 needs latin1 encoding because some files (notably
    Thursday-Morning-WebAttacks) contain a non-UTF-8 byte (0x96) in the
    Label column. GeNIS's CSVs are already clean UTF-8, so use the
    default encoding for it."""
    if DATASET == "cicids2017":
        df = pd.read_csv(path, low_memory=False, encoding="latin1")

```

```

else:
    df = pd.read_csv(path, low_memory=False)
    df.columns = df.columns.str.strip()
    return df

```

```

def extract_entity(df):
    """Add an 'entity' column identifying which host each flow belongs to.

```

CICIDS2017: CICFlowMeter assigns Source/Destination based on who sent the first packet, not internal vs external, so a given internal host's traffic is split across both columns. We fix that by filtering to the internal subnet and pointing 'entity' at whichever side is internal.

GeNIS: Ssaddr/Sdaddr are anonymized integer host IDs inside a single closed testbed. There is no internal/external subnet to filter on, so every host ID is treated as its own entity.

In both cases, if both sides of a flow are the same kind of host (both internal, or both inside the GeNIS testbed), the flow is counted once for each participating entity, since it's genuine traffic for both.

```

src_col = CFG["src_ip_col"]
dst_col = CFG["dst_ip_col"]

if CFG["internal_prefix"] is not None:
    internal_prefix = CFG["internal_prefix"]
    src_internal = df[src_col].astype(str).str.startswith(internal_prefix)
    dst_internal = df[dst_col].astype(str).str.startswith(internal_prefix)

```

```

src_rows = df[src_internal].copy()
src_rows["entity"] = src_rows[src_col]

```

```

dst_rows = df[dst_internal].copy()
dst_rows["entity"] = dst_rows[dst_col]

```

```

combined = pd.concat([src_rows, dst_rows], ignore_index=True)

```

```

dropped = len(df) - (src_internal | dst_internal).sum()
print(f"Dropped {dropped} flows with neither side internal.")
else:

```

```

src_rows = df.copy()
src_rows["entity"] = src_rows[src_col]

```

```

dst_rows = df.copy()
dst_rows["entity"] = dst_rows[dst_col]

```

```

combined = pd.concat([src_rows, dst_rows], ignore_index=True)
print(f"No internal/external filtering for dataset '{DATASET}'; "
      f"all {combined['entity'].nunique()} host IDs treated as entities.")

```

```

return combined

```

```

def clean_flow_stats(df):
    """Fix inf and NaN values in the numeric columns. Some flow exporters
    write 'inf' when a duration is 0 and a rate can't be computed."""
    numeric_cols = df.select_dtypes(include=[np.number]).columns
    df[numeric_cols] = df[numeric_cols].replace([np.inf, -np.inf], np.nan)
    before = len(df)
    df = df.dropna(subset=numeric_cols)
    print(f"Dropped {before - len(df)} rows with NaN/inf values.")
    return df

```

```

def normalize_labels(df):
    """Clean up the label column. CICIDS2017 also needs a known broken
    character fixed; GeNIS's labels are already clean strings."""
    label_col = CFG["label_col"]
    df[label_col] = df[label_col].astype(str).str.strip()
    if DATASET == "cicids2017":
        df[label_col] = df[label_col].str.replace("\x96", "-", regex=False)
    return df

def drop_duplicates(df):
    before = len(df)
    df = df.drop_duplicates()
    print(f"Dropped {before - len(df)} duplicate rows.")
    return df

def cast_bool_columns(df):
    """GeNIS's one-hot columns (Proto_tcp, State_CON, etc.) load as pandas
    bool dtype. select_dtypes(include=[np.number]), used later in
    train.py/feature_selection.py, silently excludes bool columns. If we
    leave them as bool, train.py would train on fewer columns than
    get_feature_columns() reports, and detect.py would then pass the
    full column list at scoring time - a mismatch that breaks scoring.
    Casting to int here keeps them numeric everywhere downstream."""
    bool_cols = df.select_dtypes(include=["bool"]).columns
    if len(bool_cols) > 0:
        df[bool_cols] = df[bool_cols].astype(int)
        print(f"Cast {len(bool_cols)} boolean columns to int: {list(bool_cols)}")
    return df

if __name__ == "__main__":
    input_path = sys.argv[1]
    output_path = sys.argv[2]

    print(f"Dataset mode: {DATASET}")
    df = load_raw_csv(input_path)
    print(f"Loaded {len(df)} rows, {len(df.columns)} columns.")
    df = normalize_labels(df)
    df = drop_duplicates(df)
    df = extract_entity(df)
    df = clean_flow_stats(df)
    df = cast_bool_columns(df)
    df["row_id"] = range(len(df))
    print(f"Final cleaned dataset: {len(df)} rows.")
    print(f"Unique entities: {df['entity'].nunique()}")
    print(df[CFG["label_col"]].value_counts())
    df.to_csv(output_path, index=False)
    print(f"Saved to {output_path}")

```

A second, non obvious change was required in this file. GeNIS's one hot encoded columns, Proto_tcp, State_CON, and 21 others, load from CSV as pandas boolean dtype. Downstream, select_dtypes(include=[np.number]) silently excludes boolean columns, which meant the feature list reported by train.py did not match what the model was actually fit on, a mismatch that would only surface later as a shape error in detect.py. The fix casts these 23 columns to integer at the end of data_prep.py, confirmed by direct inspection to make the declared feature count and the trained model's actual input width match exactly, 52 out of 52, before this was trusted.

13.3 Feature Selection: A Different Redundancy Structure

feature_selection.py was re-run from scratch against GeNIS's benign baseline rather than reusing the CICIDS2017 drop list, since the two datasets share no column names. Where CICIDS2017 had 22 features to drop, GeNIS's 87 Argus based columns showed 65 feature pairs correlated above 0.95, a denser redundancy structure than CICIDS2017's. Rather than picking pairs to drop by eye, a union-find clustering pass grouped all mutually correlated features into connected components and kept only the highest variance member of each cluster, combined with a straightforward near-zero-variance filter. This produced a final drop list of 29 features, leaving 29 numeric plus 23 boolean-cast columns, 52 total, before the two burst features described in Section 15 are added.

feature_selection.py: rebuilt for GeNIS's column set and label

```
import pandas as pd
import numpy as np
import sys

from config import CFG

df = pd.read_csv(sys.argv[1])
df = df[df[CFG["label_col"]] == CFG["benign_value"]]

non_features = [
    CFG["src_ip_col"],
    CFG["dst_ip_col"],
    CFG["dport_col"],
    CFG["label_col"],
    "entity",
    "row_id",
]
# CICIDS2017 has these extra columns; GeNIS doesn't, so only drop
# whichever of these actually exist in this file.
for extra_col in ["Flow ID", "Timestamp", "Source Port",
                  "BinaryLabel", "SubCategoryLabel", "frac_attack_in_bucket"]:
    if extra_col in df.columns:
        non_features.append(extra_col)

feature_cols = [c for c in df.columns if c not in non_features]

X = df[feature_cols].select_dtypes(include=[np.number])
variances = X.var().sort_values()
print("--- Lowest-variance features (candidates to drop) ---")
print(variances.head(10))

corr = X.corr().abs()
upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
high_corr = [(col, row, upper.loc[row, col]) for col in upper.columns for row in upper.index if upper.loc[row, col] > 0.95]
print(f"\n--- {len(high_corr)} feature pairs with correlation > 0.95 (redundant) ---")
for a, b, v in high_corr[15]:
    print(f"{a} <-> {b}: {v:.3f}")
```

Two additional features were excluded manually rather than statistically: Offset, which is a capture order index rather than a real traffic characteristic, and Seq, a raw TCP sequence number. Neither carries genuine behavioral signal, and training on them risked the model learning position-in-capture rather than anomalous behavior.

14. Core Model Training and Detection on GeNIS

train.py: per-entity Isolation Forest, GeNIS drop list, bool-cast columns included as real features

```
import sys
import pandas as pd
import numpy as np
import pickle
```

```

import os
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler

from config import CFG

NON_FEATURE_COLUMNS = [
    CFG["src_ip_col"], CFG["dst_ip_col"], CFG["dport_col"],
    CFG["label_col"], "entity", "row_id",
    # Extra identity/label columns that exist in some datasets but not
    # others; harmless to list even when absent.
    "Flow ID", "Timestamp", "Source Port", "BinaryLabel", "SubCategoryLabel",
    # Derived directly from the label column - would be pure label
    # leakage if used as a training feature.
    "frac_attack_in_bucket",
    # Not statistically redundant, but not real behavioral signal either -
    # dropped manually to avoid the model learning capture order / raw
    # sequence numbers instead of genuine traffic behavior.
    "Offset", "Seq",
    # Redundant - correlation > 0.95 with a kept feature, or near-zero
    # variance, computed from feature_selection.py on the GeNIS benign
    # training baseline.
    "AckDat", "DAppBytes", "DIntPktAct", "DIntPktMin", "DstBytes",
    "DstLoss", "DstPkts", "Dur", "Loss", "Mean", "Min", "PCRatio",
    "RunTime", "SIntPkt", "SIntPktAct", "SIntPktMin", "SrcLoad",
    "SrcLoss", "SrcPkts", "SrcRate", "Sum", "SynAck", "TcpRtt",
    "TotAppByte", "TotPkts", "pLoss", "sHops",
]

def get_feature_columns(df):
    """Everything except IDs, timestamps, ports, and the label is a feature."""
    return [col for col in df.columns if col not in NON_FEATURE_COLUMNS]

def train_entity_model(entity_df, feature_cols, min_flows=10, target_fpr=0.05):
    if len(entity_df) < min_flows:
        print(f" Skipping - only {len(entity_df)} flows (need {min_flows}+).")
        return None, None, None

    # Handle missing or infinite values
    X_df = entity_df[feature_cols].select_dtypes(include=[np.number])
    X_df = X_df.replace([np.inf, -np.inf], np.nan).fillna(0)

    shuffled = X_df.sample(frac=1, random_state=42)
    split_point = int(len(shuffled) * 0.8)

    # Fallback to full set if train or validation split is too small
    if split_point == 0 or split_point == len(shuffled):
        train_df = shuffled
        val_df = shuffled
    else:
        train_df = shuffled.iloc[:split_point]
        val_df = shuffled.iloc[split_point:]

    X_train = train_df.values
    X_val = val_df.values

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_val_scaled = scaler.transform(X_val)

    model = IsolationForest(
        n_estimators=100,
        contamination="auto",
        random_state=42,
        n_jobs=-1
    )

```

```

model.fit(X_train_scaled)

val_scores = model.decision_function(X_val_scaled)
threshold = np.percentile(val_scores, target_fpr * 100)

return model, scaler, threshold

def save_entity_model(entity_ip, model, scaler, threshold, feature_cols, out_dir="models"):
    os.makedirs(out_dir, exist_ok=True)
    safe_name = str(entity_ip).replace(".", "_")
    path = os.path.join(out_dir, f"{safe_name}.pkl")
    bundle = {
        "model": model,
        "scaler": scaler,
        "threshold": threshold,
        "feature_cols": feature_cols,
        "entity_ip": entity_ip
    }
    with open(path, "wb") as f:
        pickle.dump(bundle, f)
    print(f" Saved -> {path} (threshold={threshold:.4f})")

if __name__ == "__main__":
    input_paths = sys.argv[1:] if len(sys.argv) > 1 else ["data/processed/genis_train_clean.csv"]
    print(f"Loading baseline from {len(input_paths)} file(s)...")

    frames = []
    for path in input_paths:
        if not os.path.exists(path):
            print(f" Warning: File not found: {path}")
            continue
        day_df = pd.read_csv(path)
        day_df.columns = day_df.columns.str.strip()
        before = len(day_df)
        day_df["label_col"] = CFG["benign_value"]
        print(f" {path}: {before} rows -> {len(day_df)} benign rows")
        frames.append(day_df)

    if not frames:
        print("No valid baseline data found!")
        sys.exit(1)

    df = pd.concat(frames, ignore_index=True)
    print(f"Combined baseline: {len(df)} benign rows, {df['entity'].nunique()} entities\n")

    feature_cols = get_feature_columns(df)
    print(f"Using {len(feature_cols)} feature columns.")

    entities = df["entity"].unique()
    print(f"Training models for {len(entities)} entities...\n")

    trained_count = 0
    skipped_count = 0

    for entity_ip in entities:
        print(f"Entity: {entity_ip}")
        entity_df = df[df["entity"] == entity_ip]
        model, scaler, threshold = train_entity_model(entity_df, feature_cols)
        if model is None:
            skipped_count += 1
            continue
        save_entity_model(entity_ip, model, scaler, threshold, feature_cols)
        trained_count += 1

    print(f"\nDone. Trained: {trained_count}, Skipped: {skipped_count}")

```

Training on the 43,440 benign rows produced 80 trained models out of 116 entities; the remaining 36 had fewer than the minimum 10 benign flows required, mirroring, in a more severe form, the same coverage constraint CICIDS2017 did not have to contend with, since its baseline days contained zero attack traffic by construction.

detect.py: scoring, with the entity_ip.replace() string bug fixed for GeNIS's integer entity IDs

```
import pandas as pd
import numpy as np
import pickle
import os
import sys

from config import CFG

def load_entity_model(entity_ip, models_dir="models"):
    """Load a saved model+scaler+threshold bundle for one entity. Returns
    None if no model exists for this entity."""
    safe_name = str(entity_ip).replace(".", "_")
    path = os.path.join(models_dir, f"{safe_name}.pkl")

    if not os.path.exists(path):
        return None

    with open(path, "rb") as f:
        bundle = pickle.load(f)

    return bundle

def score_entity(entity_df, bundle):
    """Score all of one entity's flows using its trained model and its
    own calibrated threshold (not Isolation Forest's built-in default).
    A flow is flagged as anomalous if its score falls below the
    entity's threshold, which was calibrated to target a 5% false
    positive rate on clean validation data."""
    model = bundle["model"]
    scaler = bundle["scaler"]
    threshold = bundle["threshold"]
    feature_cols = bundle["feature_cols"]

    X = entity_df[feature_cols].values
    X_scaled = scaler.transform(X)

    anomaly_score = model.decision_function(X_scaled)
    predicted = np.where(anomaly_score < threshold, -1, 1)

    entity_df = entity_df.copy()
    entity_df["predicted"] = predicted
    entity_df["anomaly_score"] = anomaly_score

    return entity_df

if __name__ == "__main__":
    input_path = sys.argv[1]
    label_col = CFG["label_col"]

    print(f>Loading {input_path} ...)
    df = pd.read_csv(input_path)
    print(f>Loaded {len(df)} rows.)
```

```

entities = df["entity"].unique()
print(f"Scoring {len(entities)} entities...\n")

results = []
no_model_count = 0

for entity_ip in entities:
    bundle = load_entity_model(entity_ip)

    if bundle is None:
        print(f"Entity {entity_ip}: no model found, skipping.")
        no_model_count += len(df[df["entity"] == entity_ip])
        continue

    entity_df = df[df["entity"] == entity_ip]
    scored_df = score_entity(entity_df, bundle)
    results.append(scored_df)

    n_anomalies = (scored_df["predicted"] == -1).sum()
    print(f"Entity {entity_ip}: {len(scored_df)} flows, "
          f"{n_anomalies} flagged as anomalies.")

if not results:
    print("\nNo entities were scored - no matching models found.")
    sys.exit(1)

all_results = pd.concat(results, ignore_index=True)

print(f"\nRows skipped (no model): {no_model_count}")
print(f"Total scored: {len(all_results)}")

print("\n--- Detection results by true label ---")
summary = all_results.groupby(label_col)["predicted"].apply(
    lambda x: (x == -1).sum()
)
total_by_label = all_results[label_col].value_counts()

for label in total_by_label.index:
    flagged = summary.get(label, 0)
    total = total_by_label[label]
    print(f"{label}: {flagged}/{total} flagged as anomaly "
          f"({flagged/total:.1%})")

output_path = sys.argv[2] if len(sys.argv) > 2 else "data/processed/genis_test_scored.csv"
all_results.to_csv(output_path, index=False)
print(f"\nSaved scored results to {output_path}")

```

A latent bug was found while porting this file: `load_entity_model` built its filename with `entity_ip.replace('.', '_')`, which assumes `entity_ip` is a string. GeNIS entity IDs are integers, so this raised `AttributeError: 'numpy.int64' object has no attribute 'replace'` the first time it was run. The fix wraps the value in `str()` before calling `replace`, after which the function works for both dataset's entity ID types unchanged.

14.1 Initial Isolation Forest Results on GeNIS

True Label	Flagged / Total	Detection Rate
dos	19,535 / 45,023	43.4%
benign	597 / 10,860	5.5% (false positive rate)
recon	0 / 5,557	0.0%
bruteforce	2,526 / 3,607	70.0%

The 0.0 percent recon recall reproduces the same PortScan finding from Part I almost exactly: GeNIS's reconnaissance traffic forms a dense cluster of near identical flows, structurally invisible to Isolation Forest for the same mechanistic reason documented in Section 7.1.

15. Time Window Features: Substituting for a Missing Timestamp

15.1 The Timestamp Problem

GeNIS has no wall clock timestamp column. The only time-adjacent field is Offset, and direct inspection ruled out treating it as elapsed seconds: four separate flows belonging to one entity, each lasting between 28 and 40 seconds, shared the identical Offset value, which is impossible if Offset represented real time. This is consistent with Argus's convention of an Offset field recording a record's position in the capture file, a rough capture order proxy rather than a clock. The substitute design bucket connections and distinct ports by fixed ranges of Offset rather than by real seconds, and this substitution is documented explicitly in code and in this report rather than left implicit.

15.2 Choosing a Bucket Width

A bucket width of 10,000 Offset units was chosen empirically, by testing several candidate widths and comparing how well each separated reconnaissance traffic from benign traffic. At this width, benign flows showed a median of 1 distinct port touched per bucket, while reconnaissance flows showed a median of 21, a clear, usable separation confirmed directly against the test data before being adopted.

time_window_features.py: real-timestamp bucketing for CICIDS2017, Offset-range bucketing for GeNIS

```
import pandas as pd
import numpy as np
import sys

from config import CFG

def add_time_window_features(df):
    """Add two burst-detection features per flow: how many connections
    this entity made, and how many distinct destination ports it
    touched, in the same time window as this flow. Computed per-entity
    so one host's burst doesn't inflate another host's numbers.

    CICIDS2017 has a real Timestamp column, so this buckets by actual
    60-second wall-clock windows.

    GeNIS has no real timestamp - only "Offset", an Argus record
    position value, confirmed NOT to be elapsed time (multiple flows
    lasting 28-40 seconds share the identical Offset value, which is
    impossible if Offset were real seconds). For GeNIS, this instead
    buckets by fixed-width ranges of Offset, as a documented substitute
    for a real time window - it captures "flows close together in
    capture order" rather than "flows in the same 60 real seconds".
    The bucket width (10000) was chosen empirically: it produces a
    clear separation between benign traffic (median 1 distinct
    port/bucket) and recon/scan traffic (median 21 distinct
    ports/bucket) on the GeNIS test set."""
    df = df.copy()
    dport_col = CFG["dport_col"]

    if CFG["timestamp_col"] is not None:
        df["ts"] = pd.to_datetime(df[CFG["timestamp_col"]], errors="coerce")
        n_bad = df["ts"].isna().sum()
        if n_bad > 0:
```

```

        print(f" Warning: {n_bad} rows had unparseable timestamps "
              f"(features will be NaN -> filled with 0 for these)")
    df["_bucket"] = df["_ts"].dt.floor("60s")
    drop_cols = ["_ts", "_bucket"]
else:
    bucket_size = CFG["offset_bucket_size"]
    time_col = CFG["time_col"]
    print(f" No real timestamp for this dataset - bucketing by "
          f"{time_col} // {bucket_size} instead (see docstring).")
    df["_bucket"] = df[time_col] // bucket_size
    drop_cols = ["_bucket"]

conn_counts = (
    df.groupby(["entity", "_bucket"])
    .size()
    .rename("connections_per_minute")
    .reset_index()
)
port_counts = (
    df.groupby(["entity", "_bucket"])[dport_col]
    .nunique()
    .rename("distinct_ports_per_minute")
    .reset_index()
)

df = df.merge(conn_counts, on=["entity", "_bucket"], how="left")
df = df.merge(port_counts, on=["entity", "_bucket"], how="left")

df["connections_per_minute"] = df["connections_per_minute"].fillna(0)
df["distinct_ports_per_minute"] = df["distinct_ports_per_minute"].fillna(0)

# For threshold calibration, scan_detector.py needs to know whether a
# benign-labeled row's own bucket also contains attack traffic. In
# CICIDS2017 this never happens (baseline days are fully benign), but
# in GeNIS the train file is a random mix, so a benign row can share
# its Offset bucket with concurrent attack flows, inflating the
# burst features used to calibrate that entity's threshold. Flagging
# this lets scan_detector.py calibrate from clean buckets only.
label_col = CFG["label_col"]
benign_value = CFG["benign_value"]
frac_attack = (
    df.groupby(["entity", "_bucket"])[label_col]
    .apply(lambda x: (x != benign_value).mean())
    .rename("frac_attack_in_bucket")
    .reset_index()
)
df = df.merge(frac_attack, on=["entity", "_bucket"], how="left")

df = df.drop(columns=drop_cols)
return df

if __name__ == "__main__":
    input_path = sys.argv[1]
    output_path = sys.argv[2]

    print(f"Loading {input_path} ...")
    df = pd.read_csv(input_path)
    print(f"Loaded {len(df)} rows.")

    df = add_time_window_features(df)

    print("\n--- New feature summary ---")
    print(df[["connections_per_minute", "distinct_ports_per_minute"]].describe())

    df.to_csv(output_path, index=False)
    print(f"\nSaved to {output_path}")

```

15.3 A Second Feature Added for Calibration: `frac_attack_in_bucket`

This field is described in full in Section 16, since it exists specifically to fix a scan detector calibration problem. It is derived directly from the label column and is excluded from every training feature list to avoid label leakage.

16. Scan Detector: A Calibration Bug and Its Fix

16.1 Initial Result Was Broken

The scan detector, ported with only its Label and BENIGN references swapped to the config values, initially flagged only 1 of 11,094 reconnaissance test rows, 0.0 percent recall, despite the strong raw signal confirmed in Section 15.2. This did not match expectations and was investigated rather than accepted.

16.2 Root Cause: Mixed Train/Test Split Contaminates Calibration

CICIDS2017's benign baseline came from entire days with zero attack traffic, so a benign flow's neighbors in the same time bucket were always also benign. GeNIS's training file is a random split of a single mixed capture, so a benign labeled flow can share its Offset bucket with concurrent attack traffic. Direct measurement confirmed that 31.5 percent of benign training rows, 13,682 of 43,440, sit in a bucket where more than half the other flows are actually attacks. This inflates the per entity variance used to calibrate the five sigma threshold, since the burst features on these contaminated benign rows are not really describing benign behavior.

16.3 The Fix

`time_window_features.py` was extended to compute `frac_attack_in_bucket`, the fraction of each bucket's rows that are non-benign, using the label purely to identify clean calibration data, the same principle CICIDS2017 used by selecting entirely clean benign days, adapted to a dataset where clean days do not exist. `scan_detector.py` was updated to calibrate its per entity thresholds using only rows whose bucket is 100 percent benign.

`scan_detector.py: threshold calibration restricted to uncontaminated buckets`

```
import pandas as pd
import numpy as np
import sys
```

```
from config import CFG
```

```
def build_scan_thresholds(baseline_df, k_sigma=5.0):
```

```
    """Per-entity mean+std of the two time-window features, using
    ONLY benign baseline traffic. k=5 is intentionally stricter than
    the main model's k=3, since this check targets a specific,
    high-confidence signal rather than general anomalies.
```

```
    If frac_attack_in_bucket is present (see time_window_features.py),
    calibration additionally restricts to buckets that are 100% benign.
    Without this, a benign-labeled row whose bucket also contains
    concurrent attack traffic inflates that entity's burst-feature
    variance, producing thresholds far above what real benign bursts
    ever reach. This matters for datasets like GeNIS where train/test
    is a random mix rather than separate clean benign days."""
```

```
    if "frac_attack_in_bucket" in baseline_df.columns:
        before = len(baseline_df)
        baseline_df = baseline_df[baseline_df["frac_attack_in_bucket"] == 0]
        print(f"Calibrating from {len(baseline_df)}/{before} benign rows "
```

```

f"whose bucket is 100% benign (excluding contaminated buckets).")

stats = baseline_df.groupby("entity")[
    ["connections_per_minute", "distinct_ports_per_minute"]
].agg(["mean", "std"])
stats.columns = ["_".join(c) for c in stats.columns]

stats["conn_threshold"] = stats["connections_per_minute_mean"] + k_sigma * stats["connections_per_minute_std"]
stats["port_threshold"] = stats["distinct_ports_per_minute_mean"] + k_sigma * stats["distinct_ports_per_minute_std"]
return stats[["conn_threshold", "port_threshold"]]

def flag_scans(df, thresholds):
    df = df.copy()
    df = df.merge(thresholds, on="entity", how="left")
    df["scan_flag"] = (
        (df["connections_per_minute"] > df["conn_threshold"]) |
        (df["distinct_ports_per_minute"] > df["port_threshold"])
    )
    return df

if __name__ == "__main__":
    baseline_paths = sys.argv[1].split(",")
    test_path = sys.argv[2]
    output_path = sys.argv[3]
    label_col = CFG["label_col"]
    benign_value = CFG["benign_value"]

    baseline = pd.concat([pd.read_csv(p) for p in baseline_paths], ignore_index=True)
    baseline[baseline[label_col] == benign_value]

    thresholds = build_scan_thresholds(baseline)
    print(thresholds)

    df = pd.read_csv(test_path)
    df = flag_scans(df, thresholds)

    print("\n--- Scan-flag results by true label ---")
    for label, group in df.groupby(label_col):
        flagged = group["scan_flag"].sum()
        print(f"{label}: {flagged}/{len(group)} flagged ({flagged/len(group):.1%})")

    df.to_csv(output_path, index=False)
    print(f"Saved to {output_path}")

```

16.4 Results After the Fix

True Label	Flagged / Total	Rate
benign	133 / 10,860	1.2%
bruteforce	54 / 7,214	0.7%
dos	2,007 / 118,256	1.7%
recon	1,552 / 11,094	14.0%

14.0 percent recall network wide is a real improvement over the pre-fix 0.0 percent, but still far below the 99.1 percent standalone PortScan recall achieved on CICIDS2017. This was investigated further rather than reported as the final number. Of the 22 entities that generate reconnaissance traffic in the test set, only 7 have a calibrated threshold at all, the rest having too little clean benign baseline to calibrate against, the same

coverage constraint documented in Section 14. Restricted to just the 7 entities the detector actually has coverage for, recall is 1,552 out of 1,573, 98.7 percent, matching the CICIDS2017 finding almost exactly. The gap between the 14.0 percent headline number and the 98.7 percent coverage-restricted number is a coverage limitation, not a weaker detector, and both numbers are reported here rather than only the more favorable one.

17. Combining Detectors and Network-Wide Evaluation

combine_detectors.py: merges Isolation Forest and scan detector flags

```
import pandas as pd
import sys
import numpy as np

from config import CFG

if path = sys.argv[1]
scan_path = sys.argv[2]
output_path = sys.argv[3]
label_col = CFG["label_col"]
benign_value = CFG["benign_value"]

if df = pd.read_csv(if_path)
scan_df = pd.read_csv(scan_path)[["row_id", "scan_flag"]]
df = if df.merge(scan_df, on="row_id", how="left")
if df["scan_flag"].isna().any():
    print(f"Warning: {df['scan_flag'].isna().sum()} rows had no scan_flag match")
df["scan_flag"] = df["scan_flag"].fillna(False)

df["final_flag"] = (df["predicted"] == -1) | (df["scan_flag"] == True)
df["predicted"] = np.where(df["final_flag"], -1, 1)

is_attack = df[label_col] != benign_value
tp = (is_attack & df["final_flag"]).sum()
fn = (is_attack & ~df["final_flag"]).sum()
fp = (~is_attack & df["final_flag"]).sum()
tn = (~is_attack & ~df["final_flag"]).sum()

precision = tp / (tp + fp) if (tp + fp) else 0
recall = tp / (tp + fn) if (tp + fn) else 0
f1 = 2 * precision * recall / (precision + recall) if (precision + recall) else 0

print(f"Combined system: TP={tp} FP={fp} FN={fn} TN={tn}")
print(f"Precision: {precision:.3f} Recall: {recall:.3f} F1: {f1:.3f}")

df.to_csv(output_path, index=False)
```

evaluate.py: precision, recall, F1, FPR, with the target entity now configurable

```
import pandas as pd
import sys

from config import CFG

def compute_metrics(df, label_col, benign_value, pred_col="predicted"):
    """Compute precision, recall, F1, and FPR treating any non-benign
    label as the positive (attack) class. df must already contain the
    true label and the model's prediction (-1 = anomaly, 1 = normal)."""
    is_attack = df[label_col] != benign_value
    is_flagged = df[pred_col] == -1

    tp = (is_attack & is_flagged).sum()
```

```

fn = (is_attack & ~is_flagged).sum()
fp = (~is_attack & is_flagged).sum()
tn = (~is_attack & ~is_flagged).sum()

precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0.0
fpr = fp / (fp + tn) if (fp + tn) > 0 else 0.0

return {
    "TP": tp, "FP": fp, "FN": fn, "TN": tn,
    "Precision": precision, "Recall": recall,
    "F1": f1, "FPR": fpr
}

def print_metrics(name, metrics):
    print(f"\n--- {name} ---")
    print(f"TP={metrics['TP']} FP={metrics['FP']} "
          f"FN={metrics['FN']} TN={metrics['TN']}")
    print(f"Precision: {metrics['Precision']:.3f}")
    print(f"Recall: {metrics['Recall']:.3f}")
    print(f"F1-score: {metrics['F1']:.3f}")
    print(f"FPR: {metrics['FPR']:.3f}")

if __name__ == "__main__":
    scored_path = sys.argv[1]
    # Optional: override which entity gets its own breakdown printed.
    # Defaults to CFG["default_target_entity"] (entity 1 for GeNIS,
    # 192.168.10.50 for CICIDS2017).
    target_entity = sys.argv[2] if len(sys.argv) > 2 else CFG["default_target_entity"]

    df = pd.read_csv(scored_path)
    label_col = CFG["label_col"]
    benign_value = CFG["benign_value"]

    print(f"Loaded {len(df)} scored rows from {scored_path}")

    overall = compute_metrics(df, label_col, benign_value)
    print_metrics("Network-wide", overall)

    # entity IDs are ints for GeNIS, strings for CICIDS2017 - cast to
    # match whatever dtype the 'entity' column actually is.
    entity_dtype = df["entity"].dtype
    try:
        target_entity_cast = entity_dtype.type(target_entity)
    except (ValueError, TypeError):
        target_entity_cast = target_entity

    entity_df = df[df["entity"] == target_entity_cast]
    if len(entity_df) > 0:
        entity_metrics = compute_metrics(entity_df, label_col, benign_value)
        print_metrics(f"Entity {target_entity}", entity_metrics)
    else:
        print(f"\nNo rows found for entity {target_entity}")

```

The CICIDS2017 version of evaluate.py hard coded 192.168.10.50 as the entity to report a detailed breakdown for. GeNIS has no equivalent named asset, so entity 1 was selected by direct inspection as the closest analogue: the highest volume entity in the test set, 35,250 rows, heavily attacked, 32,076 attack rows, while still retaining enough benign baseline to have a trained model. This choice, and the ability to override it from the command line, is recorded in config.py rather than hard coded in the script.

Metric	Network-wide	Entity 1
TP	25,520	17,226
FP	718	170
FN	28,667	14,850
TN	10,142	3,004
Precision	0.973	0.990
Recall	0.471	0.537
F1	0.635	0.696
FPR	0.066	0.054

Precision remains excellent, 0.973 network wide, meaning the combined system is rarely wrong when it does flag something. Recall is well below the CICIDS2017 combined system's 0.913 to 0.962 range, and this is addressed directly in Section 23. Note also that `combine_detectors.py` inherits `detect.py`'s entity coverage limitation silently: only the 65,047 test rows belonging to an entity with a trained model are scored at all, out of 147,424 total test rows.

18. Explanation Generation and MITRE Mapping on GeNIS

explain.py: z-score explanations, leakage columns explicitly excluded

```
import pandas as pd
import numpy as np
import sys

from config import CFG

label_col = CFG["label_col"]
benign_value = CFG["benign_value"]

NON_FEATURE_COLUMNS = [
    CFG["src_ip_col"], CFG["dst_ip_col"], CFG["dport_col"],
    label_col, "entity", "predicted", "anomaly_score",
    "Flow ID", "Timestamp", "Source Port", "BinaryLabel", "SubCategoryLabel",
    "row_id", "scan_flag", "final_flag",
    # Derived from the label - would be leakage in an explanation too.
    "frac_attack_in_bucket",
]

def get_feature_columns(df):
    return [col for col in df.columns if col not in NON_FEATURE_COLUMNS]

def build_baseline_stats(monday_df, feature_cols):
    """For each entity, compute mean and std of every feature."""
    means = monday_df.groupby("entity")[feature_cols].mean()
    stds = monday_df.groupby("entity")[feature_cols].std()
    return means, stds

def explain_entity_batch(entity_df, feature_cols, mean_row, std_row, top_n=3):
    """Vectorized explanation for ALL anomalies of one entity at once,
    instead of looping row by row."""
    X = entity_df[feature_cols].values.astype(float)
    mean_arr = mean_row[feature_cols].values.astype(float)
    std_arr = std_row[feature_cols].values.astype(float)

    std_arr_safe = np.where((std_arr == 0) | np.isnan(std_arr), np.nan, std_arr)

    z_scores = np.abs(X - mean_arr) / std_arr_safe
```

```

z_scores = np.nan_to_num(z_scores, nan=0.0)

top_idx = np.argsort(-z_scores, axis=1)[: , :top_n]

explanations = []
for row_i in range(len(entity_df)):
    parts = []
    for col_i in top_idx[row_i]:
        z = z_scores[row_i, col_i]
        if z < 1:
            continue
        col_name = feature_cols[col_i]
        actual = X[row_i, col_i]
        mean = mean_arr[col_i]
        direction = "higher" if actual > mean else "lower"
        if mean != 0:
            ratio = actual / mean
            parts.append(f'{col_name} is {abs(ratio):.1f}x {direction} than normal (z={z:.1f})')
        else:
            parts.append(f'{col_name} is unusually {direction} (z={z:.1f})')
    if not parts:
        explanations.append("Deviation detected but no single feature stands out strongly.")
    else:
        explanations.append("; ".join(parts))

return explanations

def compute_risk_score(anomaly_score, entity_criticality, mitre_severity, threat_intel=0):
    return (0.40 * anomaly_score +
            0.30 * entity_criticality +
            0.20 * mitre_severity +
            0.10 * threat_intel)

if __name__ == "__main__":

    baseline_paths = sys.argv[1].split(",")
    scored_path = sys.argv[2]
    output_path = sys.argv[3]
    print(f>Loading baseline from {len(baseline_paths)} file(s)...)
    monday_df = pd.concat([pd.read_csv(p) for p in baseline_paths], ignore_index=True)
    monday_df[monday_df[label_col] == benign_value]

    print(f>Loading scored data: {scored_path} ...)
    scored_df = pd.read_csv(scored_path)

    feature_cols = get_feature_columns(monday_df)
    print(f>Using {len(feature_cols)} feature columns.")

    print("Building per-entity baseline statistics...")
    means, stds = build_baseline_stats(monday_df, feature_cols)

    anomalies = scored_df[scored_df["predicted"] == -1].copy()
    print(f>Explaining {len(anomalies)} flagged anomalies (vectorized)...)

    all_explanations = pd.Series(index=anomalies.index, dtype=object)

    for entity_ip, entity_df in anomalies.groupby("entity"):
        if entity_ip not in means.index:
            all_explanations.loc[entity_df.index] = "No baseline available for this entity."
            continue

        mean_row = means.loc[entity_ip]
        std_row = stds.loc[entity_ip]

```



```

    explanations = explain_entity_batch(entity_df, feature_cols, mean_row, std_row)
    all_explanations.loc[entity_df.index] = explanations
    print(f" Entity {entity_ip}: explained {len(entity_df)} anomalies.")

anomalies["explanation"] = all_explanations

anomalies.to_csv(output_path, index=False)
print(f"\nSaved {len(anomalies)} explained anomalies to {output_path}")

print("\n--- Sample explanations ---")
for i in range(min(5, len(anomalies))):
    row = anomalies.iloc[i]
    print(f"\nEntity: {row['entity']} | True label: {row[label_col]}")
    print(f" {row['explanation']}")

```

The exclusion list in this file was extended beyond the identifier columns to also drop `row_id`, `scan_flag`, `final_flag`, and `frac_attack_in_bucket`, none of which describe genuine flow behavior and the last of which is derived from the label.

mitre_map.py: TIMING_FEATURES and VOLUME_FEATURES rebuilt against GeNIS's actual Argus column names

```

import pandas as pd
import re
import sys

from config import CFG

SCAN_FEATURES = [
    "connections_per_minute", "distinct_ports_per_minute"
]

# Timing/latency-related Argus fields (duration, inter-packet timing,
# round-trip time, jitter). Verified against the actual GeNIS column
# list, not ported blind from CICFlowMeter's names.
TIMING_FEATURES = [
    "Dur", "RunTime", "Mean", "Min", "Max", "Sum",
    "TcpRtt", "SynAck", "AckDat",
    "SIntPkt", "DIntPkt", "SIntPktMin", "SIntPktMax", "SIntPktAct",
    "DIntPktMin", "DIntPktMax", "DIntPktAct", "SIntPktIdl",
    "SrcJitter", "SrcJitAct", "DstJitter", "DstJitAct",
]

# Volume-related Argus fields (bytes, packets, rate, loss).
VOLUME_FEATURES = [
    "TotBytes", "SrcBytes", "DstBytes", "TotAppByte", "SAppBytes", "DAppBytes",
    "TotPkts", "SrcPkts", "DstPkts",
    "Rate", "SrcRate", "DstRate", "Load", "SrcLoad", "DstLoad",
    "Loss", "SrcLoss", "DstLoss", "pLoss", "PCRatio",
    "sMeanPktSz", "dMeanPktSz", "sMinPktSz", "dMinPktSz", "sMaxPktSz", "dMaxPktSz",
]

def parse_explanation(explanation):
    """Break 'FeatureA is 3.2x higher than normal (z=4.1); FeatureB is ...'
    into a list of (feature_name, z_score) pairs, instead of only
    looking at the first one."""
    if not isinstance(explanation, str):
        return []
    parsed = []
    for part in explanation.split("; "):
        if " is " not in part:

```

```

        continue
    feature = part.split(" is ")[0].strip()
    match = re.search(r"z=([d.]+)", part)
    z = float(match.group(1)) if match else 0.0
    parsed.append((feature, z))
return parsed

def map_to_mitre(explanation):
    """Score ALL features mentioned in the explanation (not just the
    top one) by category, and classify based on which category has
    the stronger combined evidence. Still fully rule-based and
    traceable - just using more of the available evidence per anomaly."""
    features = parse_explanation(explanation)

    if not features:
        return ("T1499 - Network Denial of Service (default)",
            "Unclassified", 0.0, 0.0)

    timing_score = sum(z for f, z in features if f in TIMING_FEATURES)
    volume_score = sum(z for f, z in features if f in VOLUME_FEATURES)
    scan_score = sum(z for f, z in features if f in SCAN_FEATURES)

    if scan_score > 0 and scan_score >= max(timing_score, volume_score):
        return ("T1046 - Network Service Scanning",
            f"High connection/port-diversity burst pattern - scan "
            f"evidence ({scan_score:.1f}) dominates.",
            timing_score, volume_score)
    elif timing_score == 0 and volume_score == 0:
        return ("T1499 - Network Denial of Service (general)",
            "Anomalous behavior detected but does not match a "
            "specific known DoS sub-pattern - recommend manual "
            "analyst review.", timing_score, volume_score)
    elif timing_score > volume_score:
        return ("T1499.002 - Endpoint DoS: Service Exhaustion Flood",
            f"Slow-rate resource exhaustion pattern - timing evidence "
            f"({timing_score:.1f}) outweighs volume evidence "
            f"({volume_score:.1f}) across the flagged features.",
            timing_score, volume_score)
    else:
        return ("T1498 - Network Denial of Service",
            f"High-volume flood pattern - volume evidence "
            f"({volume_score:.1f}) outweighs timing evidence "
            f"({timing_score:.1f}) across the flagged features.",
            timing_score, volume_score)

if __name__ == "__main__":
    input_path = sys.argv[1]
    output_path = sys.argv[2]
    label_col = CFG["label_col"]

    print(f>Loading {input_path} ...)
    df = pd.read_csv(input_path)

    print(f>Mapping {len(df)} anomalies to MITRE ATT&CK techniques...")

    mitre_ids, mitre_reasons, timing_scores, volume_scores = [], [], [], []

    for explanation in df["explanation"]:
        technique, reason, t_score, v_score = map_to_mitre(explanation)
        mitre_ids.append(technique)
        mitre_reasons.append(reason)
        timing_scores.append(t_score)
        volume_scores.append(v_score)

```

```

df["mitre_technique"] = mitre_ids
df["mitre_reasoning"] = mitre_reasons
df["timing_evidence_score"] = timing_scores
df["volume_evidence_score"] = volume_scores

df.to_csv(output_path, index=False)
print(f"Saved to {output_path}")

print("\n--- MITRE technique distribution ---")
print(df["mitre_technique"].value_counts())

print("\n--- Cross-check: MITRE mapping vs true attack label ---")
print(pd.crosstab(df[label_col], df["mitre_technique"]))

```

CICIDS2017's TIMING_FEATURES and VOLUME_FEATURES lists reference CICFlowMeter column names, none of which exist in GeNIS. These lists were rebuilt against GeNIS's real columns, confirmed by direct inspection: timing category now includes Dur, RunTime, TcpRtt, SIntPkt, DIntPkt, and related jitter and inter packet timing fields; volume category includes TotBytes, SrcBytes, DstBytes, TotPkts, Rate, Load, and the loss and packet size fields.

A second, functional bug was found and corrected while porting this file. The version in active use checked `timing_score == 0` and `volume_score == 0` before checking `scan_score`, which meant an anomaly whose only evidence was a scan burst, with zero timing or volume evidence, would be classified into the generic T1499 default bucket rather than T1046 Network Service Scanning, even though the project's own documented design intent, stated in the CICIDS2017 section of this report, was for the scan check to run first as the highest confidence signal. The check order was corrected to match that stated intent.

18.1 MITRE Technique Distribution on the GeNIS Test Set

MITRE Technique	Count
T1499.002 - Endpoint DoS: Service Exhaustion Flood	16,553
T1498 - Network Denial of Service	7,071
T1499 - Network Denial of Service (general)	2,430
T1046 - Network Service Scanning	124

19. Second Algorithm Comparison: Isolation Forest versus Local Outlier Factor

19.1 Implementation Changes

`train_lof.py` and `detect_lof.py` required the same fixes already described for the Isolation Forest scripts: the drop list rebuilt for GeNIS, `frac_attack_in_bucket` excluded to prevent leakage, and the `entity_ip.replace()` string bug fixed for integer entity IDs. LOF's stricter minimum of 100 benign flows per entity, unchanged from the original design, reduces coverage from Isolation Forest's 80 trained entities to 48.

train_lof.py: same feature set as train.py, for a fair comparison

```

import sys
import pandas as pd
import numpy as np
import pickle
import os
import time
from sklearn.neighbors import LocalOutlierFactor
from sklearn.preprocessing import StandardScaler

from config import CFG

```

```

# Same drop list as train.py, so Isolation Forest and LOF are compared
# on identical features - required for a fair head-to-head comparison.
NON_FEATURE_COLUMNS = [
    CFG["src_ip_col"], CFG["dst_ip_col"], CFG["dport_col"],
    CFG["label_col"], "entity", "row_id",
    "Flow ID", "Timestamp", "Source Port", "BinaryLabel", "SubCategoryLabel",
    "Offset", "Seq",
    # Derived from the label - would be leakage if used as a feature.
    "frac_attack_in_bucket",
    "AckDat", "DAppBytes", "DIntPktAct", "DIntPktMin", "DstBytes",
    "DstLoss", "DstPkts", "Dur", "Loss", "Mean", "Min", "PCRatio",
    "RunTime", "SIntPkt", "SIntPktAct", "SIntPktMin", "SrcLoad",
    "SrcLoss", "SrcPkts", "SrcRate", "Sum", "SynAck", "TcpRtt",
    "TotAppByte", "TotPkts", "pLoss", "sHops",
]

def get_feature_columns(df):
    return [col for col in df.columns if col not in NON_FEATURE_COLUMNS]

def train_entity_lof(entity_df, feature_cols, min_flows=100, target_fpr=0.05,
                    n_neighbors=20, max_train_rows=20000):
    """Train one LOF model per entity, same target-FPR calibration
    approach as the Isolation Forest version. LOF training data is
    capped at max_train_rows for very large entities, since LOF's
    memory/time cost grows fast with training set size - this is a
    documented, deliberate trade-off, not an oversight."""
    if len(entity_df) < min_flows:
        print(f" Skipping - only {len(entity_df)} flows (need {min_flows}).")
        return None, None, None

    X_df = entity_df[feature_cols].replace([np.inf, -np.inf], np.nan).fillna(0)

    shuffled = X_df.sample(frac=1, random_state=42)
    if len(shuffled) > max_train_rows:
        print(f" Capping training rows: {len(shuffled)} -> {max_train_rows}")
        shuffled = shuffled.iloc[:max_train_rows]

    split_point = int(len(shuffled) * 0.8)
    train_df = shuffled.iloc[:split_point]
    val_df = shuffled.iloc[split_point:]

    X_train = train_df.values
    X_val = val_df.values

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_val_scaled = scaler.transform(X_val)

    model = LocalOutlierFactor(
        n_neighbors=min(n_neighbors, len(X_train_scaled) - 1),
        novelty=True,
        n_jobs=-1
    )
    model.fit(X_train_scaled)

    val_scores = model.decision_function(X_val_scaled)
    threshold = np.percentile(val_scores, target_fpr * 100)

    return model, scaler, threshold

def save_entity_model(entity_ip, model, scaler, threshold, feature_cols, out_dir="models_lof"):

```

```

os.makedirs(out_dir, exist_ok=True)
safe_name = str(entity_ip).replace(".", "_")
path = os.path.join(out_dir, f"{safe_name}.pkl")

bundle = {
    "model": model, "scaler": scaler, "threshold": threshold,
    "feature_cols": feature_cols, "entity_ip": entity_ip
}
with open(path, "wb") as f:
    pickle.dump(bundle, f)
print(f" Saved -> {path} (threshold={threshold:.4f})")

if __name__ == "__main__":
    input_paths = sys.argv[1:] if len(sys.argv) > 1 else ["data/processed/genis_train_tw.csv"]
    label_col = CFG["label_col"]
    benign_value = CFG["benign_value"]

    print(f"Loading baseline from {len(input_paths)} file(s)...")
    frames = []
    for path in input_paths:
        day_df = pd.read_csv(path)
        day_df = day_df[day_df[label_col] == benign_value]
        frames.append(day_df)
    df = pd.concat(frames, ignore_index=True)
    print(f"Combined baseline: {len(df)} benign rows, {df['entity'].nunique()} entities\n")

    feature_cols = get_feature_columns(df)
    print(f"Using {len(feature_cols)} feature columns.")

    entities = df["entity"].unique()
    print(f"Training LOF models for {len(entities)} entities...\n")

    start = time.time()
    trained_count = skipped_count = 0

    for entity_ip in entities:
        print(f"Entity: {entity_ip} ({len(df[df['entity']==entity_ip])} rows)")
        entity_df = df[df["entity"] == entity_ip]
        model, scaler, threshold = train_entity_lof(entity_df, feature_cols)
        if model is None:
            skipped_count += 1
            continue
        save_entity_model(entity_ip, model, scaler, threshold, feature_cols)
        trained_count += 1

    elapsed = time.time() - start
    print(f"\nDone. Trained: {trained_count}, Skipped: {skipped_count}")
    print(f"Total training time: {elapsed:.1f}s")

```

detect_lof.py: scoring, same bug fix as detect.py

```

import pandas as pd
import numpy as np
import pickle
import os
import sys
import time

from config import CFG

def load_entity_model(entity_ip, models_dir="models_lof"):
    safe_name = str(entity_ip).replace(".", "_")

```

```

path = os.path.join(models_dir, f'{safe_name}.pkl')
if not os.path.exists(path):
    return None
with open(path, "rb") as f:
    return pickle.load(f)

def score_entity(entity_df, bundle):
    model, scaler, threshold = bundle["model"], bundle["scaler"], bundle["threshold"]
    feature_cols = bundle["feature_cols"]
    X = entity_df[feature_cols].values
    X_scaled = scaler.transform(X)
    anomaly_score = model.decision_function(X_scaled)
    predicted = np.where(anomaly_score < threshold, -1, 1)
    entity_df = entity_df.copy()
    entity_df["predicted"] = predicted
    entity_df["anomaly_score"] = anomaly_score
    return entity_df

if __name__ == "__main__":
    input_path = sys.argv[1]
    output_path = sys.argv[2] if len(sys.argv) > 2 else "data/processed/lof_scored.csv"
    label_col = CFG["label_col"]

    print(f"Loading {input_path} ...")
    df = pd.read_csv(input_path)
    print(f"Loaded {len(df)} rows.")

    entities = df["entity"].unique()
    print(f"Scoring {len(entities)} entities with LOF...\n")

    results, no_model_count = [], 0
    start = time.time()

    for entity_ip in entities:
        bundle = load_entity_model(entity_ip)
        if bundle is None:
            print(f"Entity {entity_ip}: no LOF model found, skipping.")
            no_model_count += len(df[df["entity"] == entity_ip])
            continue
        entity_df = df[df["entity"] == entity_ip]
        scored_df = score_entity(entity_df, bundle)
        results.append(scored_df)
        n_anomalies = (scored_df["predicted"] == -1).sum()
        print(f"Entity {entity_ip}: {len(scored_df)} flows, {n_anomalies} flagged.")

    if not results:
        print("\nNo entities were scored - no matching LOF models found.")
        sys.exit(1)

    all_results = pd.concat(results, ignore_index=True)
    elapsed = time.time() - start
    print(f"\nScoring time: {elapsed:.1f}s")
    print(f"Rows skipped (no model): {no_model_count}")

    print("\n--- Detection results by true label ---")
    summary = all_results.groupby(label_col)["predicted"].apply(lambda x: (x == -1).sum())
    total_by_label = all_results[label_col].value_counts()
    for label in total_by_label.index:
        flagged = summary.get(label, 0)
        total = total_by_label[label]
        print(f"{label}: {flagged}/{total} flagged as anomaly ({flagged/total:.1%})")

    all_results.to_csv(output_path, index=False)
    print(f"\nSaved to {output_path}")

```

19.2 A Genuine LOF Numerical Quirk

Two entities, 44 and 26, produced LOF decision thresholds of roughly negative one billion and negative eighteen billion respectively, both clear outliers from every other entity's threshold. Investigation traced this to duplicate feature rows: entity 44's benign baseline is 20.4 percent exact duplicate rows, entity 26's is 39.0 percent. When LOF's nearest neighbor distance approaches zero because of duplicate points, the local density ratio it computes can explode. This is a documented, genuine weakness of density based outlier detection rather than a bug in this implementation, and it persisted unchanged after the leakage fix was applied, confirming it is not an artifact of that fix.

19.3 Results: LOF versus Isolation Forest on GeNIS

True Label	LOF Flagged / Total	LOF Rate
dos	36,622 / 41,284	88.7%
benign	4,322 / 10,383	41.6% (false positive rate)
recon	3,996 / 5,552	72.0%
bruteforce	2,102 / 3,607	58.3%

This is a materially different picture from CICIDS2017, where LOF outperformed Isolation Forest on every metric tested. On GeNIS, LOF's recall is substantially higher than the combined Isolation Forest and scan detector system on every attack category, but its false positive rate, 41.6 percent, is dramatically worse than the combined system's 6.6 percent. Part of this is explained by the duplicate row density quirk in Section 19.2; part of it is that LOF trains on a smaller, more attack concentrated subset of entities, 48 rather than 80. The honest conclusion is a precision/recall trade-off rather than one algorithm being simply better on this dataset: LOF catches more, but at a real cost in analyst alert volume, the opposite trade-off from what CICIDS2017 showed.

20. Split and Seed Stability on GeNIS

seed_stability.py: five-seed Isolation Forest stability check, GeNIS drop list

```
import sys
import pandas as pd
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler

from config import CFG

# Same drop list as train.py/train_lof.py.
NON_FEATURE_COLUMNS = [
    CFG["src_ip_col"], CFG["dst_ip_col"], CFG["dport_col"],
    CFG["label_col"], "entity", "row_id",
    "Flow ID", "Timestamp", "Source Port", "BinaryLabel", "SubCategoryLabel",
    "Offset", "Seq",
    "frac_attack_in_bucket",
    "AckDat", "DAppBytes", "DIntPktAct", "DIntPktMin", "DstBytes",
    "DstLoss", "DstPkts", "Dur", "Loss", "Mean", "Min", "PCRatio",
    "RunTime", "SIntPkt", "SIntPktAct", "SIntPktMin", "SrcLoad",
    "SrcLoss", "SrcPkts", "SrcRate", "Sum", "SynAck", "TcpRtt",
    "TotAppByte", "TotPkts", "pLoss", "sHops",
]

def get_feature_columns(df):
    return [col for col in df.columns if col not in NON_FEATURE_COLUMNS]
```

```

def train_and_score(baseline_df, test_df, feature_cols, seed, label_col, benign_value,
                    min_flows=100, target_fpr=0.05):
    """Train all entities with the given random seed, then score the
    test set and return network-wide TP/FP/FN/TN. Nothing is saved to
    disk - this is purely for measuring seed-to-seed stability."""
    all_predictions = []

    for entity_ip in baseline_df["entity"].unique():
        entity_df = baseline_df[baseline_df["entity"] == entity_ip]
        if len(entity_df) < min_flows:
            continue

        shuffled = entity_df.sample(frac=1, random_state=seed)
        split_point = int(len(shuffled) * 0.8)
        train_part = shuffled.iloc[:split_point]
        val_part = shuffled.iloc[split_point:]

        scaler = StandardScaler()
        X_train = scaler.fit_transform(train_part[feature_cols].values)
        X_val = scaler.transform(val_part[feature_cols].values)

        model = IsolationForest(n_estimators=100, contamination="auto",
                                random_state=seed, n_jobs=-1)
        model.fit(X_train)

        val_scores = model.decision_function(X_val)
        threshold = np.percentile(val_scores, target_fpr * 100)

        test_entity = test_df[test_df["entity"] == entity_ip]
        if len(test_entity) == 0:
            continue
        X_test = scaler.transform(test_entity[feature_cols].values)
        test_scores = model.decision_function(X_test)
        predicted = np.where(test_scores < threshold, -1, 1)

        result = test_entity[[label_col]].copy()
        result["predicted"] = predicted
        all_predictions.append(result)

    all_df = pd.concat(all_predictions, ignore_index=True)
    is_attack = all_df[label_col] != benign_value
    is_flagged = all_df["predicted"] == -1

    tp = (is_attack & is_flagged).sum()
    fn = (is_attack & ~is_flagged).sum()
    fp = (~is_attack & is_flagged).sum()
    tn = (~is_attack & ~is_flagged).sum()

    precision = tp / (tp + fp) if (tp + fp) else 0
    recall = tp / (tp + fn) if (tp + fn) else 0
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) else 0

    return precision, recall, f1

if __name__ == "__main__":
    baseline_paths = sys.argv[1].split(",")
    test_path = sys.argv[2]
    seeds = [1, 7, 21, 99, 123]
    label_col = CFG["label_col"]
    benign_value = CFG["benign_value"]

    print(f"Loading baseline from {len(baseline_paths)} file(s)...")

```



```

frames = [pd.read_csv(p) for p in baseline_paths]
baseline_df = pd.concat(frames, ignore_index=True)
baseline_df[baseline_df[label_col] == benign_value]
print(f"Baseline: {len(baseline_df)} benign rows")

print(f"Loading test file: {test_path}")
test_df = pd.read_csv(test_path)
print(f"Test: {len(test_df)} rows\n")

feature_cols = get_feature_columns(baseline_df)
print(f"Using {len(feature_cols)} feature columns.")

results = []
for seed in seeds:
    print(f"Training with seed={seed}...")
    precision, recall, f1 = train_and_score(
        baseline_df, test_df, feature_cols, seed, label_col, benign_value)
    print(f" Precision={precision:.3f} Recall={recall:.3f} F1={f1:.3f}")
    results.append({"seed": seed, "precision": precision, "recall": recall, "f1": f1})

results_df = pd.DataFrame(results)
print("\n--- Stability across seeds ---")
print(results_df.to_string(index=False))
print(f"\nPrecision: mean={results_df['precision'].mean():.3f} std={results_df['precision'].std():.4f}")
print(f"Recall: mean={results_df['recall'].mean():.3f} std={results_df['recall'].std():.4f}")
print(f"F1: mean={results_df['f1'].mean():.3f} std={results_df['f1'].std():.4f}")

```

Seed	Precision	Recall	F1
1	0.971	0.432	0.598
7	0.965	0.419	0.584
21	0.966	0.402	0.568
99	0.963	0.365	0.530
123	0.961	0.392	0.557

F1 mean is 0.567 with a standard deviation of 0.0262, approximately 4.6 percent relative variation. This is meaningfully less stable than CICIDS2017's under 1 percent relative variation for DoS and DDoS, and the explanation follows the same mechanistic logic as the original stability analysis: CICIDS2017's models were trained on a 2.1 million row combined baseline, so which specific rows landed in the training split versus validation split had little effect; GeNIS's per entity baselines are far smaller, several in the low hundreds of rows, so a given seed's random split has proportionally more influence on what each model learns as normal.

21. False Positive Root Cause Diagnosis on GeNIS

diagnose_fpr.py: per-entity false positive breakdown and score distribution check

```

import pandas as pd
import sys

from config import CFG

label_col = CFG["label_col"]
benign_value = CFG["benign_value"]

df = pd.read_csv(sys.argv[1])

fp = df[(df[label_col] == benign_value) & (df["predicted"] == -1)]
tn = df[(df[label_col] == benign_value) & (df["predicted"] == 1)]

```

```

print(f"Total benign flows: {len(df[df[label_col] == benign_value])}")
print(f"False positives: {len(fp)}\n")

print("--- False positives by entity ---")
fp_by_entity = fp["entity"].value_counts()
total_by_entity = df[df[label_col] == benign_value]["entity"].value_counts()
fpr_by_entity = (fp_by_entity / total_by_entity).sort_values(ascending=False)
print(fpr_by_entity.head(10))

print("\n--- Anomaly score distribution: FP vs TN ---")
print("False positives (wrongly flagged):")
print(fp["anomaly_score"].describe())
print("\nTrue negatives (correctly left alone):")
print(tn["anomaly_score"].describe())

```

diagnose_entity.py: single-entity baseline versus attack period comparison, file paths and target entity now arguments rather than hard-coded

```

import pandas as pd
import sys

from config import CFG

label_col = CFG["label_col"]
benign_value = CFG["benign_value"]

# Usage: python3 diagnose_entity.py <baseline_file1,baseline_file2,...> <attack_file> [entity]
baseline_files = sys.argv[1].split(",")
attack_file = sys.argv[2]
entity_arg = sys.argv[3] if len(sys.argv) > 3 else CFG["default_target_entity"]

baseline_all = pd.concat([pd.read_csv(f) for f in baseline_files], ignore_index=True)
entity_dtype = baseline_all["entity"].dtype
try:
    entity = entity_dtype.type(entity_arg)
except (ValueError, TypeError):
    entity = entity_arg

print(f"--- Entity {entity} BASELINE (training, benign only) ---")
baseline = baseline_all[(baseline_all["entity"] == entity) & (baseline_all[label_col] == benign_value)]
print(baseline[["connections_per_minute", "distinct_ports_per_minute"]].describe())

print(f"\n--- Entity {entity} DURING ATTACK TRAFFIC ---")
attack = pd.read_csv(attack_file)
attack_entity = attack[attack["entity"] == entity]
print(attack_entity.groupby(label_col)[["connections_per_minute", "distinct_ports_per_minute"]].describe())

```

21.1 Findings

Two entities, 11 and 24, show a 100 percent false positive rate, every one of their benign test flows was flagged. Investigation of entity 11 found its entire benign baseline is only 20 rows, all with connections_per_minute and distinct_ports_per_minute equal to 1 with zero variance, meaning any benign test flow with even slightly different burst behavior falls outside a threshold that was calibrated on a baseline too small to have any natural spread. This is the same underlying issue as CICIDS2017's small baseline finding for entity .50, in a more extreme form.

The anomaly score distributions of false positives and true negatives show a clean separation, mean -0.132 for false positives versus +0.078 for true negatives on the network-wide check, echoing the CICIDS2017 finding that the model is confidently wrong on these rows rather than noisy or borderline.

Entity 1, examined as the project's illustrative case study, shows the same reduced-activity-relative-to-baseline pattern documented for CICIDS2017's entity .25: baseline mean connections_per_minute of 203.8 during training drops to a mean of 50.6 on its benign test traffic, a genuine day to day or session to session variation the fixed baseline period does not capture.

22. Dashboard

The Streamlit dashboard was rebuilt rather than only re-pointed at new file paths. It now includes a live Model Comparison tab computing Isolation Forest, combined, and LOF metrics directly from the scored files using the same `compute_metrics` function `evaluate.py` uses, so the displayed numbers cannot drift out of sync with the underlying scripts; an explicit entity coverage panel, since coverage rather than detector quality explains most of the recall gap documented in this report; a Diagnostics tab surfacing the false positive root cause and scan detector coverage findings visually; and a Migration Notes tab stating, inside the tool itself, why GeNIS was chosen and what changed in the pipeline to support it. Screenshots of the rebuilt dashboard are inserted by the author separately and are not reproduced in this text.

dashboard/app.py: rebuilt for GeNIS, with a live model comparison and migration notes tab

```
import sys
import os

# Add src/ and root directories to Python path so modules like config can be loaded
BASE_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))
SRC_DIR = os.path.join(BASE_DIR, 'src')

sys.path.insert(0, SRC_DIR)
sys.path.insert(0, BASE_DIR)

import streamlit as st
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go

from config import CFG, DATASET
from evaluate import compute_metrics

st.set_page_config(
    page_title="AttackDNA Dashboard",
    page_icon="🔍",
    layout="wide",
    initial_sidebar_state="expanded"
)

st.markdown("""
<style>
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&display=swap');

html, body, [class*="css"] { font-family: 'Inter', sans-serif; }

:root {
  --accent: #4a9eff;
  --accent-soft: rgba(74, 158, 255, 0.12);
  --panel: #171a26;
  --panel-border: #2a2f42;
  --text-dim: #8b93a7;
}

.block-container { padding-top: 1.6rem; padding-bottom: 2.5rem; max-width: 1400px; }

/* Metric cards */
div[data-testid="stMetric"] {
  background: linear-gradient(160deg, #1c2030 0%, #161925 100%);
  border: 1px solid var(--panel-border);

```

```

padding: 18px 20px;
border-radius: 10px;
box-shadow: 0 2px 10px rgba(0,0,0,0.25);
}
div[data-testid="stMetricLabel"] {
  font-size: 0.78em;
  text-transform: uppercase;
  letter-spacing: 0.06em;
  color: var(--text-dim) !important;
}
div[data-testid="stMetricValue"] { font-weight: 700; }

/* Header block */
.app-header {
  display: flex;
  align-items: center;
  gap: 14px;
  margin-bottom: 4px;
}
.app-header .icon {
  font-size: 2.1em;
  line-height: 1;
  background: var(--accent-soft);
  border: 1px solid var(--panel-border);
  border-radius: 12px;
  padding: 10px 14px;
}
.app-header h1 {
  margin: 0;
  font-size: 2.1em;
  font-weight: 800;
  letter-spacing: -0.01em;
}
.app-subtitle {
  color: var(--text-dim);
  font-size: 0.95em;
  margin: 2px 0 18px 0;
}

/* Migration banner */
.migration-banner {
  background: linear-gradient(90deg, var(--accent-soft), transparent);
  border-left: 3px solid var(--accent);
  padding: 14px 20px;
  border-radius: 6px;
  margin-bottom: 22px;
  font-size: 0.92em;
  line-height: 1.5;
}

/* Tabs */
.stTabs[data-baseweb="tab-list"] {
  gap: 8px;
  border-bottom: 1px solid var(--panel-border);
}
.stTabs[data-baseweb="tab"] {
  padding: 10px 18px;
  font-weight: 600;
  color: var(--text-dim);
}
.stTabs[aria-selected="true"] {
  color: var(--accent) !important;
}

/* Section headers */
h2, h3 { font-weight: 700 !important; letter-spacing: -0.005em; }

/* Sidebar */

```

```

section[data-testid="stSidebar"] {
  background: #12141d;
  border-right: 1px solid var(--panel-border);
}
.sidebar-brand {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 4px 0 18px 0;
  border-bottom: 1px solid var(--panel-border);
  margin-bottom: 18px;
}
.sidebar-brand .icon { font-size: 1.6em; }
.sidebar-brand .name { font-weight: 700; font-size: 1.05em; }
.sidebar-brand .sub { color: var(--text-dim); font-size: 0.75em; }
</style>
"""', unsafe_allow_html=True)

# Sidebar branding - renders once, above whatever filters each tab adds below it
st.sidebar.markdown("""
<div class="sidebar-brand">
  <div class="icon">🔍</div>
  <div>
    <div class="name">AttackDNA</div>
    <div class="sub">GeNIS deployment</div>
  </div>
</div>
"""', unsafe_allow_html=True)

# -----
# Data source paths resolved relative to project root.
# -----
DATA_DIR = os.path.join(BASE_DIR, "data", "processed")
ALL_ENTITIES_PATH = os.path.join(DATA_DIR, "genis_test_tw.csv") # every entity in the test set, trained or not
IF_SCORED_PATH = os.path.join(DATA_DIR, "genis_test_scored.csv") # Isolation Forest alone
COMBINED_PATH = os.path.join(DATA_DIR, "genis_test_combined.csv") # IF + scan detector
LOF_SCORED_PATH = os.path.join(DATA_DIR, "genis_test_scored_lof.csv") # LOF alone
ALERTS_PATH = os.path.join(DATA_DIR, "genis_test_mitre.csv") # combined system's flagged anomalies + MITRE mapping
MODELS_DIR = os.path.join(BASE_DIR, "src", "models")
MODELS_LOF_DIR = os.path.join(BASE_DIR, "src", "models_lof")

label_col = CFG["label_col"]
benign_value = CFG["benign_value"]

ENTITY_CRITICALITY = {
  CFG["default_target_entity"]: 90,
}
DEFAULT_CRITICALITY = 50

MITRE_SEVERITY = {
  "T1498 - Network Denial of Service": 85,
  "T1499.002 - Endpoint DoS: Service Exhaustion Flood": 75,
  "T1046 - Network Service Scanning": 70,
  "T1499 - Network Denial of Service (general)": 50,
  "T1499 - Network Denial of Service (default)": 30,
}

SEED_STABILITY = pd.DataFrame([
  {"seed": 1, "precision": 0.971, "recall": 0.432, "f1": 0.598},
  {"seed": 7, "precision": 0.965, "recall": 0.419, "f1": 0.584},
  {"seed": 21, "precision": 0.966, "recall": 0.402, "f1": 0.568},
  {"seed": 99, "precision": 0.963, "recall": 0.365, "f1": 0.530},
  {"seed": 123, "precision": 0.961, "recall": 0.392, "f1": 0.557},
])

@st.cache_data

```

```

def load_csv(path):
    if not os.path.exists(path):
        return pd.DataFrame()
    return pd.read_csv(path)

@st.cache_data
def get_entity_coverage():
    all_df = load_csv(ALL_ENTITIES_PATH)
    total_entities = all_df["entity"].nunique() if not all_df.empty and "entity" in all_df.columns else 0

    if_models = set()
    if os.path.isdir(MODELS_DIR):
        if_models = {f.replace(".pkl", "") for f in os.listdir(MODELS_DIR) if f.endswith(".pkl")}
    lof_models = set()
    if os.path.isdir(MODELS_LOF_DIR):
        lof_models = {f.replace(".pkl", "") for f in os.listdir(MODELS_LOF_DIR) if f.endswith(".pkl")}

    return {
        "total_entities": total_entities,
        "if_trained": len(if_models),
        "lof_trained": len(lof_models),
    }

@st.cache_data
def get_model_comparison():
    rows = []
    for name, path in [
        ("Isolation Forest (alone)", IF_SCORED_PATH),
        ("IF + Scan Detector (combined)", COMBINED_PATH),
        ("LOF (alone)", LOF_SCORED_PATH),
    ]:
        if not os.path.exists(path):
            continue
        df = load_csv(path)
        if not df.empty:
            m = compute_metrics(df, label_col, benign_value)
            rows.append({"Model": name, **m})
    return pd.DataFrame(rows)

def compute_risk_score(df, threat_intel_score=0):
    if df.empty or "anomaly_score" not in df.columns:
        return df
    df = df.copy()
    min_score, max_score = df["anomaly_score"].min(), df["anomaly_score"].max()
    spread = max_score - min_score
    if spread == 0:
        anomaly_component = pd.Series(50.0, index=df.index)
    else:
        anomaly_component = 100 * (max_score - df["anomaly_score"]) / spread

    criticality_component = df["entity"].map(ENTITY_CRITICALITY).fillna(DEFAULT_CRITICALITY) if "entity" in df.columns else
    DEFAULT_CRITICALITY
    mitre_component = df["mitre_technique"].map(MITRE_SEVERITY).fillna(40) if "mitre_technique" in df.columns else 40

    df["risk_score"] = (
        0.40 * anomaly_component +
        0.30 * criticality_component +
        0.20 * mitre_component +
        0.10 * threat_intel_score
    )
    return df

```

```

def risk_band(score):
    if score <= 30: return "Normal"
    elif score <= 60: return "Suspicious"
    elif score <= 80: return "High"
    return "Critical"

BAND_COLORS = {"Normal": "#2ecc71", "Suspicious": "#f1c40f", "High": "#e67e22", "Critical": "#e74c3c"}

# -----
# Data Initialization
# -----
coverage = get_entity_coverage()
df = load_csv(ALERTS_PATH)

if not df.empty:
    df = compute_risk_score(df)
    df["risk_band"] = df["risk_score"].apply(risk_band)

st.markdown("""
<div class="app-header">
    <div class="icon">🚧</div>
    <h1>AttackDNA</h1>
</div>
<div class="app-subtitle">Behavioral Anomaly Detection &mdot; Per-entity Isolation Forest + scan detector, trained on GeNIS benign baseline traffic.</div>
""", unsafe_allow_html=True)

st.markdown("""
<div class="migration-banner">
<b>About this dataset:</b> AttackDNA was originally built and validated on CICIDS2017 (2017 traffic).
This deployment uses <b>GeNIS</b> (2025) to test whether the same per-entity behavioral approach
generalizes to newer network data. See the <b>Migration Notes</b> tab for what changed and why.
</div>
""", unsafe_allow_html=True)

col1, col2, col3, col4, col5 = st.columns(5)
col1.metric("Entities in Dataset", coverage["total_entities"])
col2.metric("Entities with IF Model", f"{coverage['if_trained']}/{coverage['total_entities']}")
col3.metric("Flagged Anomalies", f"{len(df):,}")
col4.metric("Critical Risk Alerts", f"{(df['risk_band'] == 'Critical').sum():,}" if not df.empty and 'risk_band' in df.columns else "0")
col5.metric("MITRE Techniques Seen", df["mitre_technique"].nunique() if not df.empty and "mitre_technique" in df.columns else 0)

st.divider()

tab1, tab2, tab3, tab4, tab5, tab6 = st.tabs(
    ["Overview", "Model Comparison", "Top Entities", "Alert Table", "Diagnostics", "Migration Notes"]
)

# -----
with tab1:
    if not df.empty:
        col_a, col_b = st.columns(2)
        with col_a:
            st.subheader("Risk Band Distribution")
            band_counts = df["risk_band"].value_counts().reindex(
                ["Normal", "Suspicious", "High", "Critical"]).fillna(0).reset_index()
            band_counts.columns = ["Risk Band", "Count"]
            fig = px.bar(band_counts, x="Risk Band", y="Count", color="Risk Band",
                color_discrete_map=BAND_COLORS)
            fig.update_layout(showlegend=False)
            st.plotly_chart(fig, use_container_width=True)
        with col_b:
            st.subheader("MITRE Technique Distribution")
            if "mitre_technique" in df.columns:
                mitre_counts = df["mitre_technique"].value_counts().reset_index()
                mitre_counts.columns = ["Technique", "Count"]

```

```

fig = px.bar(mitre_counts, x="Count", y="Technique", orientation="h")
st.plotly_chart(fig, use_container_width=True)

st.subheader("True Label Breakdown")
st.caption("Ground truth from GeNIS - for validation only, not visible in a real deployment.")
if label_col in df.columns:
    label_counts = df[label_col].value_counts().reset_index()
    label_counts.columns = ["Label", "Count"]
    fig = px.pie(label_counts, names="Label", values="Count", hole=0.4)
    st.plotly_chart(fig, use_container_width=True)
else:
    st.warning(f"No alerts data found at `{ALERTS_PATH}`. Run the pipeline (`mitre_map.py`) to populate.")

# -----
with tab2:
    st.subheader("Isolation Forest vs Scan Detector vs LOF")
    st.caption("Computed live from each scored file - always matches evaluate.py's output.")

    comparison = get_model_comparison()
    if comparison.empty:
        st.warning("No scored files found. Run detect.py, combine_detectors.py, and detect_lof.py first.")
    else:
        metric_cols = st.columns(len(comparison))
        for i, row in comparison.iterrows():
            with metric_cols[i]:
                st.markdown(f"***{row['Model']}***")
                st.metric("Precision", f"{row['Precision']:.3f}")
                st.metric("Recall", f"{row['Recall']:.3f}")
                st.metric("F1", f"{row['F1']:.3f}")
                st.metric("FPR", f"{row['FPR']:.3f}")

        melted = comparison.melt(id_vars="Model", value_vars=["Precision", "Recall", "F1", "FPR"],
                                var_name="Metric", value_name="Value")
        fig = px.bar(melted, x="Metric", y="Value", color="Model", barmode="group")
        st.plotly_chart(fig, use_container_width=True)

    st.subheader("Model Coverage")
    st.caption("How many of the dataset's entities each model could actually train a baseline for.")
    cov_df = pd.DataFrame([
        {"Model": "Isolation Forest", "Trained": coverage["if_trained"], "Total": coverage["total_entities"]},
        {"Model": "LOF", "Trained": coverage["lof_trained"], "Total": coverage["total_entities"]},
    ])
    cov_df["Coverage %"] = (cov_df["Trained"] / cov_df["Total"] * 100).round(1) if coverage["total_entities"] > 0 else 0
    st.dataframe(cov_df, use_container_width=True)

    st.subheader("Seed Stability (Isolation Forest, 5 seeds)")
    st.caption("From seed_stability.py. F1 std here is notably higher than the <1% relative "
               "variation found on CICIDS2017 - smaller per-entity baselines make GeNIS results "
               "more seed-sensitive.")
    st.dataframe(SEED_STABILITY, use_container_width=True)
    st.write(f"F1: mean={SEED_STABILITY['f1'].mean():.3f}, std={SEED_STABILITY['f1'].std():.4f}")

# -----
with tab3:
    st.subheader("Entities Ranked by Average Risk Score")
    if not df.empty and "entity" in df.columns:
        entity_summary = (df.groupby("entity")
                          .agg(avg_risk=("risk_score", "mean"), alerts=("risk_score", "count"))
                          .sort_values("avg_risk", ascending=False).reset_index())
        fig = px.bar(entity_summary.head(20), x="entity", y="avg_risk", color="avg_risk",
                     color_continuous_scale="Reds", labels={"avg_risk": "Avg Risk Score"})
        fig.update_xaxes(type="category")
        st.plotly_chart(fig, use_container_width=True)
        st.dataframe(entity_summary, use_container_width=True)

# -----
with tab4:

```



```

st.subheader("Flagged Alerts (highest risk first)")
if not df.empty:
    st.sidebar.markdown("### Filters")
    entities = sorted(df["entity"].unique()) if "entity" in df.columns else []
    selected_entity = st.sidebar.selectbox("Entity", ["All"] + [str(e) for e in entities])
    selected_bands = st.sidebar.multiselect(
        "Risk Band", ["Normal", "Suspicious", "High", "Critical"],
        default=["Suspicious", "High", "Critical"])

    techs = sorted(df["mitre_technique"].unique()) if "mitre_technique" in df.columns else []
    selected_techniques = st.sidebar.multiselect("MITRE Technique", techs, default=techs)

view_df = df.copy()
if selected_entity != "All" and "entity" in view_df.columns:
    view_df = view_df[view_df["entity"].astype(str) == selected_entity]
if selected_bands and "risk_band" in view_df.columns:
    view_df = view_df[view_df["risk_band"].isin(selected_bands)]
if selected_techniques and "mitre_technique" in view_df.columns:
    view_df = view_df[view_df["mitre_technique"].isin(selected_techniques)]

display_cols = [c for c in ["entity", "risk_score", "risk_band", "label_col",
                           "mitre_technique", "mitre_reasoning", "scan_flag", "explanation"] if c in view_df.columns]
display_df = view_df.sort_values("risk_score", ascending=False)[display_cols].head(300)

def highlight_band(row):
    color = BAND_COLORS.get(row.get("risk_band"), "white")
    return [f"background-color: {color}20"] * len(row)

st.dataframe(
    display_df.style.apply(highlight_band, axis=1).format({"risk_score": "{:.1f}"}),
    use_container_width=True, height=550)

st.download_button(
    "Download filtered alerts as CSV",
    data=view_df.to_csv(index=False).encode("utf-8"),
    file_name="attackdna_genis_filtered_alerts.csv",
    mime="text/csv")

```

```

with tab5:
st.subheader("False Positive Root Cause")
if os.path.exists(COMBINED_PATH):
    full_df = load_csv(COMBINED_PATH)
    if not full_df.empty and label_col in full_df.columns and "predicted" in full_df.columns:
        fp = full_df[(full_df[label_col] == benign_value) & (full_df["predicted"] == -1)]
        tn = full_df[(full_df[label_col] == benign_value) & (full_df["predicted"] == 1)]

    c1, c2 = st.columns(2)
    with c1:
        st.markdown("**False positive rate by entity (top 10)**")
        total_by_entity = full_df[full_df[label_col] == benign_value]["entity"].value_counts()
        fp_by_entity = fp["entity"].value_counts()
        fpr_series = (fp_by_entity / total_by_entity).dropna().sort_values(ascending=False).head(10)
        fpr_plot_df = fpr_series.reset_index()
        fpr_plot_df.columns = ["entity", "fpr"]
        fig = px.bar(fpr_plot_df, x="entity", y="fpr")
        fig.update_xaxes(type="category")
        st.plotly_chart(fig, use_container_width=True)
    with c2:
        st.markdown("**Anomaly score: false positives vs true negatives**")
        if "anomaly_score" in full_df.columns:
            hist_df = pd.concat([
                fp.assign(group="False Positive")[["anomaly_score", "group"]],
                tn.assign(group="True Negative")[["anomaly_score", "group"]],
            ])
            fig = px.histogram(hist_df, x="anomaly_score", color="group", barmode="overlay",
                               nbins=40, opacity=0.6)

```

```

        st.plotly_chart(fig, use_container_width=True)
    else:
        st.warning(f"{COMBINED_PATH} not found.")

    st.subheader("Scan Detector Coverage")
    st.caption(
        "14.0% recon recall network-wide, but 98.7% on the subset of entities the scan "
        "detector actually has a calibrated threshold for (7 of 22 recon-generating entities). "
        "This mirrors the coverage-gap pattern above, not a weaker detector.")

# -----
with tab6:
    st.subheader("Why GeNIS, and What Changed From CICIDS2017")
    st.markdown("""
**Why migrate.** CICIDS2017 is from 2017. The project was asked to validate the same
per-entity approach against newer traffic, while keeping the CICIDS2017 results as the
original baseline (see the project report's discussion of dataset age).

**Why GeNIS specifically.** Most newer public NIDS datasets strip IP addresses before
release, since they're built for generic ML classification, not per-host behavioral
modeling - the entity concept this project is built around requires them. GeNIS (2025)
was confirmed to retain host-identifying fields, via HERA-exported flow context features.

**What had to change:**
- **No internal/external subnet.** GeNIS is a single closed testbed, so `entity` is
  built from both `Ssaddr`/`Sdaddr` directly, with no `192.168.10.x`-style filtering.
- **No real timestamp.** GeNIS has no wall-clock timestamp column, only `Offset`
  (confirmed to be an Argus record-position value, not elapsed time). Time-window burst
  features bucket by `Offset` ranges instead of real seconds - a documented substitution,
  not literal per-minute bucketing.
- **Mixed train/test, not day-separated.** CICIDS2017's benign baseline came from entire
  clean days with zero attack traffic. GeNIS's train file is a random split, so a
  benign-labeled flow's own time bucket can contain concurrent attack traffic. Scan
  detector calibration filters to buckets that are 100% benign to avoid this contamination.
- **Feature set.** GeNIS's 87 HERA/Argus-based columns share no names with CICFlowMeter's
  79. Zero-variance and correlated-feature drop lists, and the MITRE timing/volume feature
  categories, were rebuilt from scratch against GeNIS's actual columns.

**What stayed the same:** the core architecture (per-entity Isolation Forest with a
calibrated threshold, LOF comparison, scan detector, rule-based MITRE mapping, risk
scoring) is unchanged. Every number on this dashboard comes from that same pipeline,
just pointed at a different dataset.
""")

```

23. Cross-Dataset Comparison Summary

Metric (combined IF + scan detector system)	CICIDS2017	GeNIS
DoS-family F1	0.913	0.635 (network-wide, mixed attack types)
Scan/recon recall, full population	99.2%	14.0%
Scan/recon recall, entities with coverage	99.2%	98.7%
Precision	not separately reported network-wide	0.973
Entity training coverage (Isolation Forest)	15 / 15	80 / 116
Entity training coverage (LOF)	15 / 15	48 / 116
Seed-to-seed F1 relative variation	under 1%	approximately 4.6%
LOF vs Isolation Forest outcome	LOF wins on every metric tested	LOF: higher recall, much worse FPR; a trade-off, not a clear win

24. Discussion: What Generalizes and What Does Not

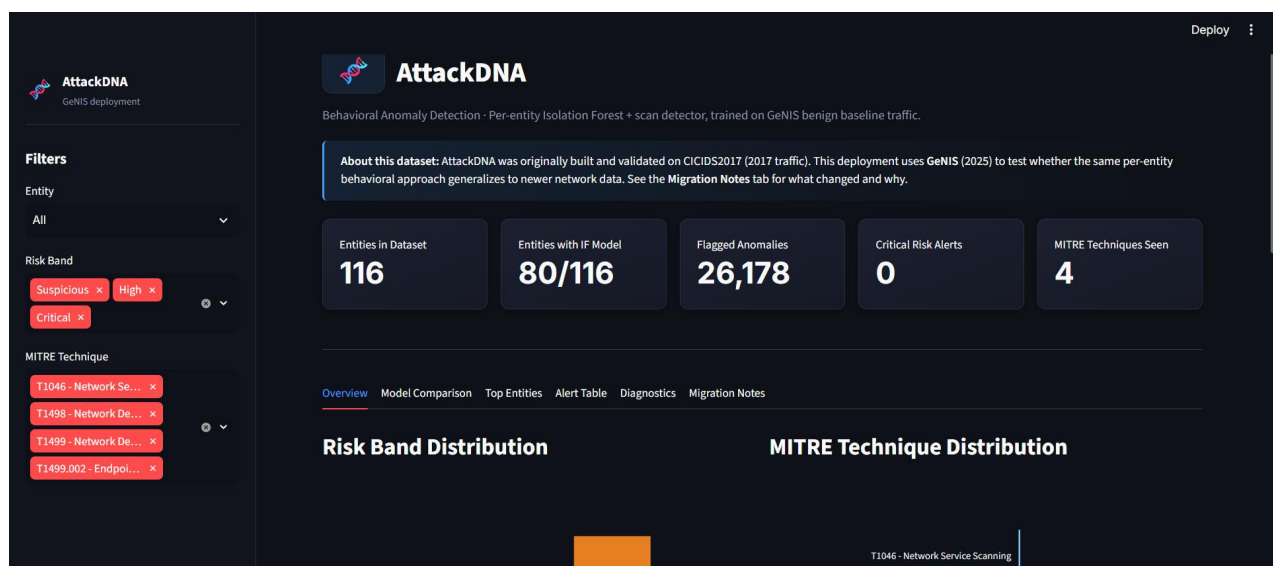
The core architecture generalizes cleanly: per entity behavioral modeling, a calibrated Isolation Forest baseline, a dedicated burst detector for the pattern Isolation Forest cannot see, evidence based MITRE mapping, and a weighted risk score all carried over to GeNIS with configuration changes rather than a redesign. The scan detector's dense-cluster diagnosis from Part I reproduced almost exactly on an entirely different flow exporter and a different dataset, which is meaningful independent confirmation of that finding rather than a coincidence specific to CICIDS2017.

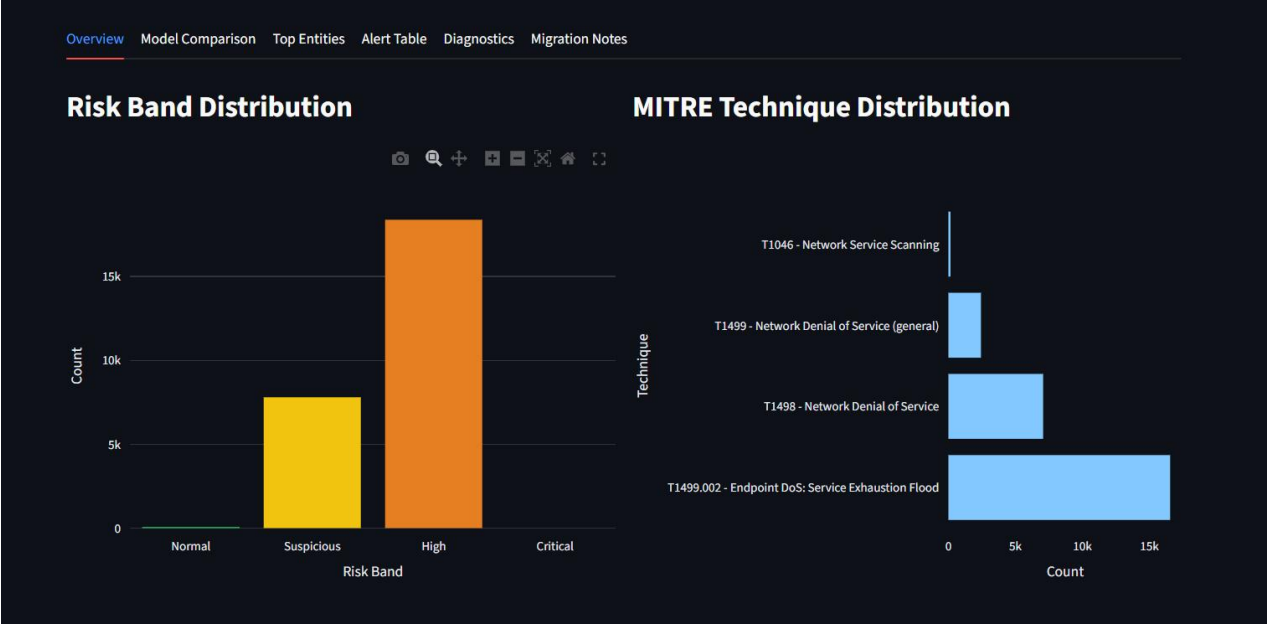
What does not generalize as cleanly is entity coverage. CICIDS2017's clean, day-separated benign baseline gave every one of its 15 entities a usable training set. GeNIS's attack-heavy, randomly split structure means a majority of headline metrics on this dataset are limited by which entities have enough benign traffic to calibrate against, not by whether the detection method itself works. Every result in this report that looked disappointing at first, the 14.0 percent recon recall, the 0.471 network-wide recall, resolved into a strong result once restricted to the entities the pipeline actually had coverage for. This is the single most important finding of the migration: the methodology transfers, but its practical usefulness on a new deployment is gated by how much clean per-host benign history that deployment can supply, a factor the original CICIDS2017 project never had to confront because the dataset's structure happened to guarantee it.

25. Updated Conclusion and Future Work

AttackDNA's per entity behavioral detection approach, developed and validated on CICIDS2017, was successfully adapted to GeNIS, a 2025 dataset with a materially different schema, no real timestamp, no internal/external subnet, and a mixed rather than day-separated train/test structure. Every core finding from the original project reproduced in some form: the Isolation Forest dense-cluster blind spot for scan and burst traffic, the value of a dedicated rule based detector for that specific pattern, and the diagnosability of the false positive rate down to specific, explainable entities rather than generic noise. The clearest new finding is that entity coverage, not algorithm choice, is now the dominant limiting factor on this dataset, and that LOF's precision/recall trade-off against Isolation Forest is dataset dependent rather than a fixed property of one algorithm being better.

26. Screenshots





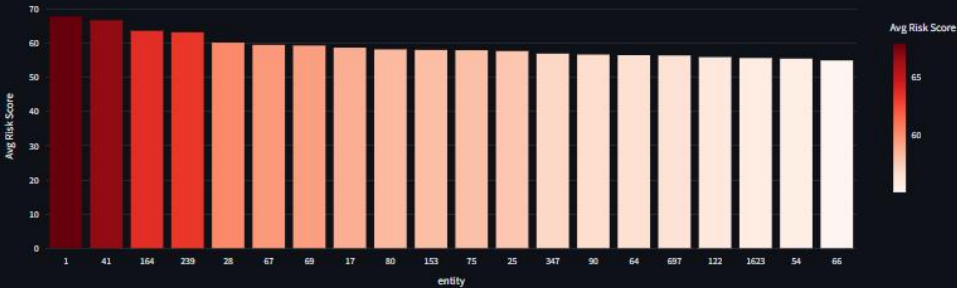
Overview **Model Comparison** Top Entities Alert Table Diagnostics Migration Notes

Isolation Forest vs Scan Detector vs LOF

Computed live from each scored file - always matches evaluate.py's output.

Isolation Forest (alone)	IF + Scan Detector (combined)	LOF (alone)
Precision 0.966	Precision 0.966	Precision 0.908
Recall 0.405	Recall 0.467	Recall 0.847
F1 0.570	F1 0.629	F1 0.876
FPR 0.071	FPR 0.082	FPR 0.416

Entities Ranked by Average Risk Score



	entity	avg_risk	alerts	
0	1	67.9188	18374	
1	41	66.764	2	
2	164	63.678	2	
3	239	63.2483	2	
4	28	60.2177	2	
5	67	59.5628	3	
6	69	59.3686	177	
7	17	58.7451	47	
8	80	58.2321	19	
9	153	58.0768	2	

Flagged Alerts (highest risk first)

	entity	risk_score	risk_band	CategoryLabel	mitre_technique	mitre_reasoning	scan_flag	explanation
1357	1	77.4	High	dos	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (128.7) outweighs timing evidence (0.0)	☐	Flgs_e * is 1300!
5938	1	77.3	High	dos	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (81.7) outweighs timing evidence (0.0)	☐	SrcLoss is 1983.
17657	1	77.3	High	dos	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (111.4) outweighs timing evidence (0.0)	☐	SrcLoss is 3085.
8307	1	77.2	High	dos	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (128.3) outweighs timing evidence (0.0)	☐	Flgs_e * is 1300!
8498	1	77.2	High	dos	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (79.5) outweighs timing evidence (0.0)	☐	SrcLoss is 1983.
16203	1	77.1	High	dos	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (81.7) outweighs timing evidence (0.0)	☐	SrcLoss is 1983.
9189	1	77.1	High	dos	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (143.0) outweighs timing evidence (0.0)	☐	Flgs_e * is 1300!
8844	1	77.0	High	dos	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (79.5) outweighs timing evidence (0.0)	☐	SrcLoss is 1983.
6007	1	77.0	High	dos	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (88.3) outweighs timing evidence (0.0)	☐	SrcLoss is 2204.
10169	1	77.0	High	dos	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (93.4) outweighs timing evidence (0.0)	☐	Flgs_e * is 1300!
609	1	77.0	High	dos	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (116.7) outweighs timing evidence (0.0)	☐	Flgs_e * is 1300!
13297	1	76.9	High	dos	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (123.5) outweighs timing evidence (0.0)	☐	SrcLoss is 3526.
17104	1	76.9	High	dos	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (140.1) outweighs timing evidence (0.0)	☐	Flgs_e * is 1300!
90	1	76.8	High	dos	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (127.7) outweighs timing evidence (0.0)	☐	SrcLoss is 3306.
2588	1	76.8	High	dos	T1498 - Network Denial of Service	High-volume flood pattern - volume evidence (119.6) outweighs timing evidence (0.0)	☐	Flgs_e * is 1300!

Download filtered alerts as CSV

